How Data Scales in Agentic Reinforcement Learning: Laws and Synthesis Strategies

Bowei He[1,2], **Yankai Chen**[1,2], **Xiaokun Zhang**[3], **Changjiang Han**[1], **Ye Yuan**[2], **Chong Li**[4], **Xue Liu**[1,2]
[1]MBZUAI, [2]McGill University, [3]CityUHK, [4]Columbia University
Bowei.He@mbzuai.ac.ae

Abstract

Pretraining scaling laws treat training data as a one-dimensional quantity: a token count. Agentic reinforcement learning (RL) inherits this scalar abstraction, but modern synthesis pipelines now generate *tasks*, *environments*, and *trajectories* independently and at very different costs, turning data into a structured design space. This raises a question pretraining never had to answer: given a synthesis budget, *which axis* should be scaled? We address this question end-to-end with three tightly coupled contributions. First, we decompose agentic data into three cardinality axes (task, environment, trajectory), plus a reward-density control variable, and fit per-axis scaling relations on different large language models (LLMs) across math+code, SWE, web, and UI/tool application domains. Across all model sizes, model families, domains, and RL algorithms (PPO and GRPO) we measured, the scaling exponents order as environment $>$ task $>$ trajectory. When environment diversity is decomposed into world and environment-structure diversity, its advantage is mainly driven by the former, while the latter is just comparable to the task diversity. Second, we use the fitted laws as a common ruler to price synthesis methods, capturing each method's efficiency as a per-axis discount factor and identifying critical synthetic-to-real ratios at which collapse begins; these ratios differ by an order of magnitude across axes. Third, we cast budget allocation as a constrained optimization under our laws and discounts, deriving a Chinchilla-style compute-optimal recipe and validating it at held-out scale. The empirical results demonstrate that following the recipe yields up to $1.7\times$ compute efficiency over the dominant "scale-the-trajectories" practice.

1 Introduction

Existing compute-optimal scaling laws for language model training typically reduce the training data distribution to a single scalar, the number of tokens, as exemplified by the Chinchilla scaling law [16] and its predecessor framework [19]. Recent agentic reinforcement learning (RL) works [13, 20, 30] inherited this single-axis abstraction, plotting performance against rollout tokens, prompt count, or training steps as if they were interchangeable. But agentic data is fundamentally different. A single "data point" simultaneously encodes a task specification, an environment with its API surface and dynamics, a sampled trajectory of model actions and tool feedback, and a reward signal, each generated by a different pipeline at a different cost. Synthesis pipelines amplify this heterogeneity: environment generators [4, 14, 50], task generators [36, 59], and trajectory generators [2, 10] act on different axes and saturate at different rates. The scalar abstraction has quietly become wrong.

This shift even changes what the central question is. With modern synthesis tooling, agentic data is effectively unbounded in supply, so the practitioner's problem is no longer "how much more data should I collect," but "where should I spend my synthesis budget?" Figure 1 makes this concrete: two training runs on Qwen3-8B with identical total tokens and FLOPs, differing only in how the

40th Conference on Neural Information Processing Systems (NeurIPS 2026).

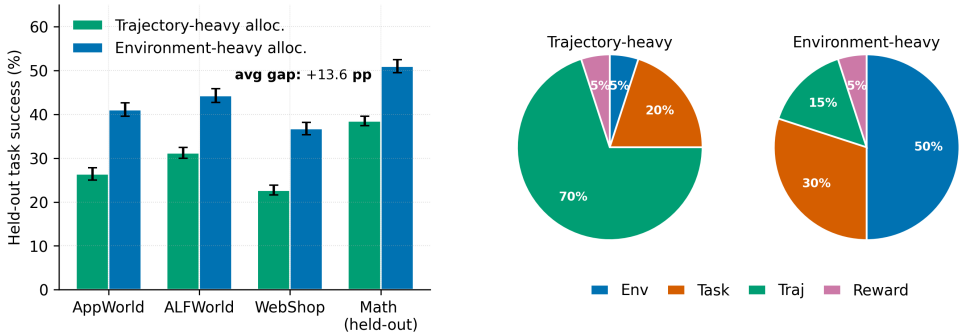


Figure 1: **Same compute, very different outcomes.** Two GRPO runs on Qwen3-8B with identical total rollout tokens and gradient FLOPs, differing only in how the data budget is split across the task, environment, and trajectory axes and the reward control. **Left:** task success rate on AppWorld, ALFWorld, WebShop, and a held-out math suite. **Right:** the corresponding allocation pies.

budget is split across (task, environment, trajectory), reach success rates that differ by 14 percentage points. A single-axis power law cannot fit both, which means the bottleneck has moved from quantity to allocation. However, allocation in current agentic RL pipelines is largely decided by convention or convenience, not by any quantitative account of what each axis is worth.

A principled answer to "where to synthesize" requires three conceptual moves, each enabling the next: (i) *decompose* the data axis into independent design dimensions and measure each independently; (ii) *price* synthesis methods by how efficiently they convert compute into "real-equivalent" data on each dimension; (iii) *solve* budget allocation as a constrained optimization under the fitted laws and discounts. The order is not optional: without per-axis exponents there is no common unit in which to price synthesis, and without prices there is no objective for the optimization to minimize.

Concretely, we argue that in agentic RL, synthesis turns the data axis from a scalar into a design space, and the right scaling laws turn that design space into a solvable optimization problem. We instantiate this argument with three contributions, each enabling the next:

- *Decomposing the design space (Section 4).* We separate agentic data into three cardinality axes, task, environment, and trajectory, plus a reward-density control variable, and fit independent scaling exponents on each axis. Across Qwen3-4B/8B/14B, GRPO and PPO, five domains, and two further model families (twenty one settings in total), we observe a robust ordering: environment $\alpha_{\mathrm{E}} >$ task $\alpha_{\mathrm{T}} >$ trajectory α_{τ}; resolving the environment axis into world and environment-structure diversity sharpens it to world $\alpha_{\mathrm{W}} >$ environment structure $\alpha_{\mathrm{V}} \approx$ task $\alpha_{\mathrm{T}} >$ trajectory α_{τ}.
- *Pricing synthesis along each axis (Section 5).* Using the fitted laws as a common ruler, we measure how efficiently existing synthesis methods convert compute into real-equivalent data, capturing this as a per-axis discount $\kappa \in [0, 1]$ and identifying critical synthetic-to-real ratios $r^{\star}$ at which collapse begins. We find $r^{\star}_{\mathrm{E}} \approx 9.4 \gg r^{\star}_{\mathrm{T}} \approx 3.4 \gg r^{\star}_{\tau} \approx 1.4$, an ordering we link mechanistically to how grounded each axis is to external truth.
- *Solving the allocation problem (Section 6).* With laws and discounts in hand, "what should I synthesize" becomes a constrained optimization with a tractable solution. We derive a compute-optimal recipe that specifies how each data-axis budget D and model size N should scale with the total compute budget C: $D^{\star}_{\mathrm{E}} \propto C^{0.34}$, $D^{\star}_{\mathrm{T}} \propto C^{0.27}$, $D^{\star}_{\tau} \propto C^{0.13}$, $N^{\star} \propto C^{0.26}$, and validate the predicted allocation at a held-out compute scale. Whether the budget is allocated to generating data or building new worlds, trajectory resampling ranks last.

Together, these contributions reframe agentic data scaling: the bottleneck is not data quantity but data allocation across a structured design space.

2 Related Work

Data scaling laws. In LLM pretraining, Kaplan et al. [19] and Hoffmann et al. [16] established power-law relationships between loss, parameters, and tokens, giving rise to compute-optimal training;

subsequent work has refined these laws under inference constraints [38], robustness checks [34], and parameter-counting interpretations [39]. In RL for LLMs, DeepSeek-R1 [13] demonstrated that pure RL with verifiable rewards scales reasoning capability, ScaleRL [20] characterized RL compute scaling under a fixed recipe, Mai et al. [30] reported a ZeroTIR-specific "Agent RL Scaling Law", and Tan et al. [47] provided an empirical study of RL post-training scaling. In both settings, data enters as a single axis: tokens in pretraining, and rollout tokens or unique prompts in RL. Rather than adding a new global axis, we decompose the data axis itself into task, environment, and trajectory axes, and find that this decomposition is necessary to predict allocation outcomes.

Agentic data synthesis. Recent work synthesizes individual axes of agentic data: environments [4, 14, 50, 56], tasks [36, 48, 59], and trajectories [2, 10, 22, 57]. A growing body of work has documented synthesis-induced quality degradation, or model collapse, in pretraining [7, 9, 41, 46, 54]. Existing methods scale a chosen axis in isolation; we provide a framework that prices them on a common scale, finding their collapse ratios differ by an order of magnitude.

Curriculum and unsupervised environment design. GenEnv [14], self-evolving curricula [3], and difficulty-aligned task generation [36, 56] address a question our framework deliberately leaves open: *in what order* should data along a given axis be presented? Our decomposition operates one level up, asking how much budget each axis deserves; curriculum operates within an axis, deciding which slice to sample at each training step. The two are complementary: our recipe specifies the per-axis budget, and curriculum methods can then schedule consumption of that budget.

3 Preliminaries and Setup

3.1 Agentic RL and the data tensor

A policy π_θ acts in an environment instance $e \in \mathcal{S}_E$ on a task $t \in \mathcal{S}_T$, producing a trajectory $\tau = (o_0, a_0, o_1, a_1, \ldots, o_H)$, where a_i are tool or action calls and o_i are observations, including environment feedback. A reward function ρ assigns $R(\tau, t, e) = \rho(\tau, t, e)$. We define the training corpus as $\mathcal{D} = \big(\mathcal{S}_T, \mathcal{S}_E, \{(t_i, e_i, \tau_i)\}_{i=1}^{n}, \rho\big)$, where $\mathcal{S}_T$ is the finite set of training tasks, $\mathcal{S}_E$ is the finite set of training environment instances, and each trajectory τ_i is associated with a task-environment pair (t_i, e_i). For later use, let $n_{t,e} = \big|\{\, i : t_i = t,\ e_i = e \,\}\big|, \Omega = \{(t,e) : n_{t,e} > 0\}$, where $n_{t,e}$ is the number of trajectories collected for cell (t, e) and Ω is the set of covered task-environment cells.

The data tensor induces three cardinality axes and one control variable. **(1) Task axis D_{T}:** the number of unique training task specifications, $D_{\mathrm{T}} = |\mathcal{S}_T|$. Task specifications are deduplicated by sentence-embedding cosine distance > 0.15; five alternative rules, including embedding-free ones, keep α_{T} within $[0.238, 0.261]$ (Appendix G). **(2) Environment axis D_{E}:** the number of distinct training environment instances, $D_{\mathrm{E}} = |\mathcal{S}_E|$. An environment-structure instance is identified by its structural signature, a canonical hash over the typed tool/API surface, the state schema, the transition semantics, and the verifier interface; content never changes it, so a procedural seed creates a new instance only if it changes the signature. The signature is a deduplication criterion, not a claim that two instances with distinct signatures constitute distinct worlds; variation in the underlying world, meaning a distinct repository, corpus, app, or site, is reported separately as world diversity D_{W} (Table 1; Appendix B). **(3) Trajectory axis D_τ:** the average number of rollouts per covered task-environment cell, $D_\tau = \frac{1}{|\Omega|} \sum_{(t,e)\in\Omega} n_{t,e}$. Increasing D_τ corresponds to repeated sampling within already covered task-environment cells, which is the standard practice in GRPO [43]. In the balanced full-factorial case, where every $(t, e) \in \mathcal{S}_T \times \mathcal{S}_E$ has the same number of rollouts, this reduces to $D_\tau = \frac{n}{D_{\mathrm{T}} D_{\mathrm{E}}}$. **(4) Reward control variable D_{R}:** the information density of the reward signal, used as a categorical sparse/dense control rather than a cardinality axis: sparse rewards provide only an outcome-level signal, whereas dense rewards provide at least three verifiable intermediate signals per trajectory (Appendix H).

3.2 Experimental testbeds

To stress-test the law's domain invariance, we use four domains: (a) **Math+code (TIR)**: Python interpreter as tool, training on synthetic math problems following the ZeroTIR recipe [23, 30], evaluated on AIME24/25 [28], MATH500 [15], AMC23 [29]. (b) **Code agents (SWE)**: SWE-Gym [33]-style execution environments, evaluated on SWE-bench Verified [18]. (c) **Web agents**:

Table 1: **Three levels of environment variation** (Section 4.4). Only L1 and L2 create new instances.

Level	What varies	Example	Exponent
L0: content	seed or initial state within the same repository and harness	a new AIME problem; a new issue on the same repository	0.07 ± 0.02 (placebo)
L1: structure	tool/API surface, state schema, transition semantics, verifier	stateless vs. persistent REPL; pytest vs. tox output grammar	$\alpha_{\mathrm{V}} = 0.28 \pm 0.03$ (0.41 on math+code)
L2: world (D_{W})	repository, corpus, app, or site with its own state	a repository or app unseen in training	$\alpha_{\mathrm{W}} = 0.44 \pm 0.04$

WebArena-Lite [51, 58] with the BrowserGym [6] interface. (d) **UI/tool agents**: AppWorld [48] and ALFWorld [45]. A fifth domain, (e) **search agents** (Wikipedia retrieval plus a paginated web-search API; Bamboogle [35] and a multi-hop split), serves as a robustness check. Evaluation is world-held-out on SWE and AppWorld but task-held-out on web (Appendix A). For axes that require synthesis, we use AWM [50] for environments, AutoPlay [36]/SCA [59] for tasks, and standard policy-rollout for trajectories.

Models and training. We use Qwen3-4B / 8B / 14B base checkpoints [53] as the size sweep, with Qwen3-8B as the default, plus Llama-3.1-8B [11] and Qwen2.5-7B [52]. **GRPO** [43] (default; group size 8, no critic) and **PPO** [40] (with a separate critic of equal size; details in Appendix A) are employed as the agentic reinforcement learning algorithms. All experiments use the verl framework [44] with VerlTool [17] for multi-tool rollout. We log full per-axis sweeps; all reported numbers are means over 3 seeds with bootstrap 95% confidence intervals (CIs).

Loss-equivalent metric. Because RL with verifiable rewards has no smooth "loss," we use $L \equiv -\log p_{\mathrm{success}}$ where p_{success} is task success rate on a held-out evaluation set, following [20, 47]. This makes our laws directly comparable to pretraining laws and allows clean log-linear fits (Appendix F).

4 Decomposing the Design Space: Per-Axis Scaling Laws

Before deciding where to synthesize data, we first quantify the marginal scaling behavior of each axis. This section implements the first of the three steps outlined in Section 1: we decompose agentic data into its three cardinality axes and fit an independent scaling exponent for each.

Functional form. To make the three cardinality axes comparable, we need a functional form in which each axis contributes its own scaling term and its own exponent. Following the additive power-law tradition of pretraining scaling laws [16, 19], we posit:

$$L(D_{\mathrm{T}}, D_{\mathrm{E}}, D_\tau) - L_\infty \;=\; A \cdot D_{\mathrm{T}}^{-\alpha_{\mathrm{T}}} + B \cdot D_{\mathrm{E}}^{-\alpha_{\mathrm{E}}} + C \cdot D_\tau^{-\alpha_\tau} \;+\; I(D_{\mathrm{T}}, D_{\mathrm{E}}, D_\tau), \qquad (1)$$

where L_∞ is the irreducible loss and I captures cross-axis interactions that we measure separately in Section 4.2. The independent exponents α_i are the loss-reduction-per-log-unit of each axis, a unit-normalized currency we use throughout the paper to compare and price synthesis methods. We call such fitted per-axis scaling relations laws for brevity; we claim the ordering of their exponents, not their values.

4.1 Independent-axis exponents

Protocol. For each axis, we hold the other two at values fixed in advance by a calibration sweep (doubling an axis changes success by less than 2 points), $(D_{\mathrm{E}}, D_{\mathrm{T}}, D_\tau) = (256, 10{,}240, 8)$, reused unchanged in every cell (Appendix A), and sweep the target axis over 6 nested grid points spanning $32\times$, extended to 8 points as a stability check. Each sweep is repeated for Qwen3-4B/8B/14B in the math+code domain. The sweeps match training signal: every D_{E} and D_{T} grid point uses the same 256,000 task-environment cells (8 rollouts each) and 1,000 updates, so rollout tokens drift by under 4% and per-environment repetition even falls $32\times$ along D_{E}; only D_τ scales tokens ($32\times$), and equalizing tokens or FLOPs lowers α_τ to 0.07 to 0.08 while moving α_{E} and α_{T} by at most 0.01 (Appendix D.1). We fit $L = KD^{-\alpha} + L_\infty$ by nonlinear weighted least squares, profiling L_∞ over a grid and solving the inner log-linear problem in closed form (Algorithm 1).

Result. Figure 2 shows clean power-law fits ($R^2 > 0.97$ on every panel) with the following fitted exponents (Qwen3-8B, math+code; full table in Appendix D):

$$\alpha_{\mathrm{E}} = 0.41 \pm 0.03, \quad \alpha_{\mathrm{T}} = 0.26 \pm 0.02, \quad \alpha_\tau = 0.11 \pm 0.02. \qquad (2)$$

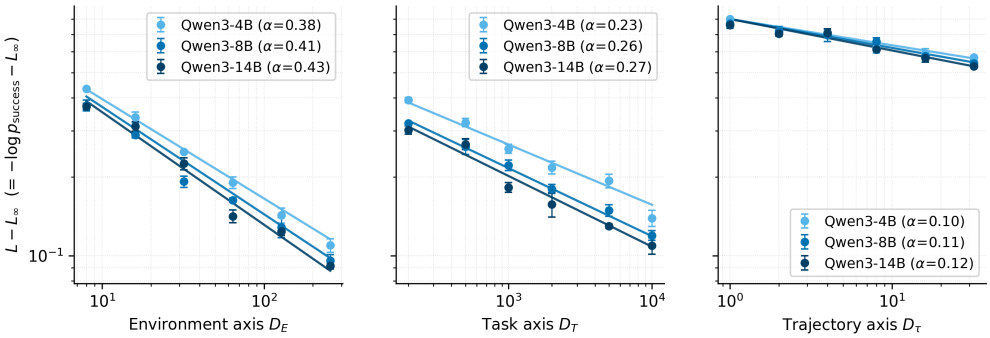


Figure 2: **Per-axis scaling laws under controlled sweeps.** Each panel sweeps one axis with the other two held at saturation; symbols are mean±SE over 3 seeds, lines are weighted log-log fits. Across all three model sizes, environment scaling is the steepest, trajectory scaling the flattest. The exponent ordering $\alpha_{\mathrm{E}} > \alpha_{\mathrm{T}} > \alpha_\tau$ holds with non-overlapping 95% CIs at every model size.

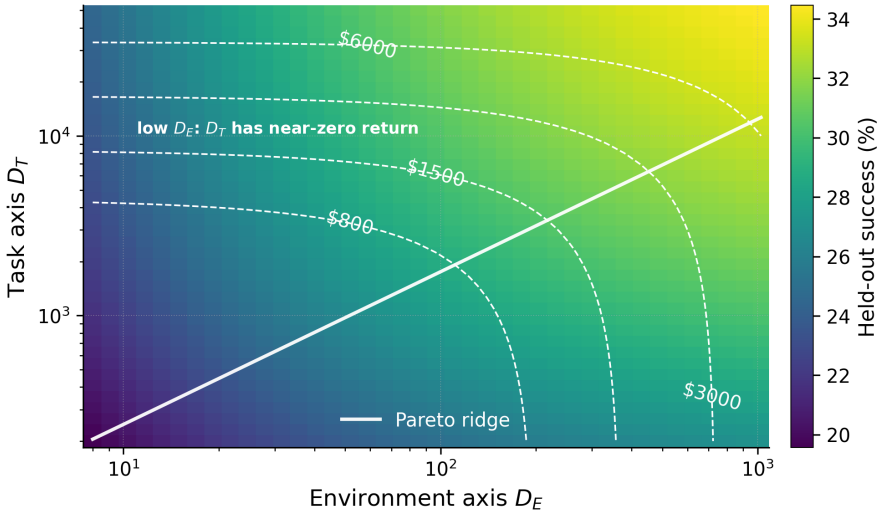


Figure 3: **Cross-axis interaction.** Heatmap of held-out success on Qwen3-8B (math+code) over a $(D_{\mathrm{E}}, D_{\mathrm{T}})$ grid at fixed $D_\tau = 8$. Iso-cost contours (dashed) reveal a Pareto ridge: at low D_{E}, scaling D_{T} has near-zero return. Environment diversity is a prerequisite for task diversity to matter.

The ordering $\alpha_{\mathrm{E}} > \alpha_{\mathrm{T}} > \alpha_\tau$ holds with non-overlapping CIs at every model size we tested. It also holds without any fit, as local log-log slopes never overlap across axes (49 of 49 comparisons; Mann-Whitney [31] $p = 2.9 \times 10^{-4}$), and paired bootstrap differences are $\alpha_{\mathrm{E}} - \alpha_{\mathrm{T}} = 0.148$ $[0.109, 0.187]$ and $\alpha_{\mathrm{T}} - \alpha_\tau = 0.150$ $[0.116, 0.184]$ (Appendix D).

Interpretation. Environment diversity is the highest-leverage axis because each new environment forces the policy to generalize across different state spaces, action vocabularies, and tool-feedback dynamics, which are structural variation forms that cannot be obtained from new tasks within a fixed environment; Section 4.4 locates most of this leverage in world diversity. Task diversity is intermediate: more tasks expose new goal structures, but the underlying interaction grammar is fixed. Trajectory resampling has the lowest exponent because additional rollouts on already-covered (task, env) pairs primarily reduce variance within an already-explored region of policy space; once a sample is informative enough to estimate advantage well, further rollouts give diminishing returns [43, 55].

4.2 Cross-axis interactions

Pure independence (Eq. 1 with $I \equiv 0$) is a first-order approximation: tasks are bound to compatible environments (9 to 41 per task on average; Appendix B), and I captures the residual coupling. Figure 3 shows that at low D_{E}, scaling D_{T} yields almost no gain, because the surface bends sharply rather than being separable. Fitting a multiplicative correction $I = -\gamma \cdot (D_{\mathrm{E}}\, D_{\mathrm{T}})^{-\beta}$ improves R^2 from 0.93 to 0.98 with $\gamma = 0.18, \beta = 0.21$. The substantive scientific takeaway is the prerequisite structure:

Table 2: **Fitted exponents across model sizes, algorithms, and domains.** Numbers are mean $\pm$ bootstrap 95% CI half-width over 3 seeds. Note the ordering $\alpha_{\mathrm{E}} > \alpha_{\mathrm{T}} > \alpha_{\tau}$ holds in all 18 settings. †No separable world axis: α_{E} is a structure exponent (Section 4.4).

Model	Algo.	Domain	α_{E}	α_{T}	α_{τ}
Qwen3-4B	GRPO	Math+Code†	0.38 ± 0.04	0.23 ± 0.03	0.10 ± 0.02
	GRPO	SWE	0.36 ± 0.05	0.21 ± 0.04	0.09 ± 0.03
	GRPO	Web	0.43 ± 0.04	0.26 ± 0.03	0.12 ± 0.02
	PPO	Math+Code†	0.35 ± 0.04	0.21 ± 0.03	0.10 ± 0.02
Qwen3-8B	GRPO	Math+Code†	0.41 ± 0.03	0.26 ± 0.02	0.11 ± 0.02
	GRPO	SWE	0.39 ± 0.03	0.24 ± 0.02	0.10 ± 0.02
	GRPO	UI/Tool	0.45 ± 0.03	0.27 ± 0.02	0.13 ± 0.02
	PPO	Math+Code†	0.39 ± 0.03	0.24 ± 0.02	0.11 ± 0.02
Qwen3-14B	GRPO	Math+Code†	0.43 ± 0.03	0.27 ± 0.02	0.12 ± 0.02
	GRPO	Web	0.46 ± 0.03	0.28 ± 0.02	0.13 ± 0.02

environment diversity gates the value of task diversity. Practically, this means that pipelines that pour task synthesis into a small environment pool (e.g., many tasks in τ-bench [1] or its small-scale variants) will see early saturation regardless of how many tasks they generate.

4.3 Robustness across model sizes, algorithms, domains, and model families

Table 2 reports the fitted exponents; across all 18 (model$\times$algo$\times$domain) cells we measured, the ordering $\alpha_{\mathrm{E}} > \alpha_{\mathrm{T}} > \alpha_{\tau}$ holds with non-overlapping CIs. Absolute exponents shift modestly: web/UI domains have larger α_{E} (more diverse tools amplify the value of environment generalization), and PPO/GRPO yield similar exponents (within 1 SE), suggesting the ordering is an algorithm-invariant property of agentic data, not an artifact of GRPO's variance-reduction mechanics.

Other families and a search domain. Reduced 4-point sweeps preserve the ordering on Llama-3.1-8B $(0.36, 0.23, 0.09)$, Qwen2.5-7B $(0.38, 0.24, 0.10)$, and the search domain $(0.42, 0.25, 0.12)$: it holds in all 21 settings while absolute values shift by up to 0.05, so the constants are regime-specific (Appendix D).

Reward control variable. Switching from sparse (outcome only) to dense (process-shaped) rewards [25, 42] on the same data corpus yields a $1.4\times$ to $2.1\times$ effective multiplier on D_{τ} but does not change α_{E} or α_{T} (Appendix H). We therefore treat D_{R} as a control variable, claim no comparable reward scaling law, and fold its multiplier into an effective trajectory count in Section 6.

4.4 Resolving the environment axis: world versus structure

Outside math+code, our pools grow the number of distinct worlds W (repositories, apps, sites) and of structural variants per world V together, $D_{\mathrm{E}} = W \times V$; the 128 SWE instances, for example, span 11 repositories. Math+code has a single world (a Python tool server), so its α_{E} is a pure structure exponent († in Table 2). Holding D_{E}, compute, and training signal fixed and moving budget from V to W, from $(W, V) = (1, 64)$ to $(8, 8)$ ($(4, 16)$ on web), raises held-out-world success at every step in SWE, UI/tool, and web, with non-overlapping CIs (e.g., from 18.0 to 26.1% on SWE; Appendix C). A joint fit gives $\alpha_{\mathrm{W}} = 0.44 \pm 0.04$ and $\alpha_{\mathrm{V}} = 0.28 \pm 0.03$ (a repository-only SWE sweep gives $\alpha_{\mathrm{W}} = 0.46 \pm 0.05$), so wherever the two separate,

$$\alpha_{\mathrm{W}}\,(0.44) \;>\; \alpha_{\mathrm{V}}\,(0.28) \;\approx\; \alpha_{\mathrm{T}}\,(0.24 \text{ to } 0.27) \;>\; \alpha_{\tau}\,(0.11).$$

World diversity is the steepest axis we measured, while structure and task diversity are not separable at our resolution: environment diversity beats task diversity through its world component. The blended α_{E} is the composition-weighted mixture of α_{W} and α_{V} (on SWE, $0.87 \times 0.44 + 0.13 \times 0.28 \approx 0.42$ against the measured 0.39 ± 0.03), and without a world axis, structure carries the whole generalization burden ($\alpha_{\mathrm{V}} = 0.41$ on math+code). Content-only variation is not environment diversity (seeds at a fixed signature give 0.07 ± 0.02), and structural diversity keeps paying beyond factor-level coverage up to an inventory ceiling of 384 signatures (Appendix C).

Table 3: **Synthesis taxonomy along the data axes.** Rows: representative synthesis methods, all run through one common harness (Appendix I). Columns: which axis each primarily scales, the effective discount κ measured at $r = D_s/D_r = 1$ (equal real and synthetic), the axis-specific collapse ratio $r^\star$, and rough cost per "unit" (1k tasks, 1 environment, or 1 trajectory) on Qwen3-8B. The estimator of Section 5.2 gives $r^\star = 9.4$, 3.4, 1.4 for AWM, AutoPlay, SWiRL.

Method	Primary axis	κ@r=1	$r^\star$	Cost/unit
AWM [50]	D_{E}	0.71 ± 0.05	9.4 ± 0.9	4.10 USD/env
GenEnv [14]	D_{E}	0.66 ± 0.06	8.1 ± 1.0	3.80 USD/env
DreamGym [4]	$D_{\mathrm{E}} + D_\tau$	0.59 ± 0.05	6.2 ± 0.8	0.04 USD/traj
AutoPlay [36]	D_{T}	0.54 ± 0.04	3.1 ± 0.4	0.18 USD/task
SCA [59]	D_{T}	0.51 ± 0.05	2.8 ± 0.4	0.21 USD/task
RLVE [56]	D_{T}	0.57 ± 0.04	3.4 ± 0.4	0.15 USD/task
SWiRL [10]	D_τ	0.34 ± 0.06	1.4 ± 0.3	0.02 USD/traj
TDScaling [2]	D_τ	0.42 ± 0.05	1.7 ± 0.3	0.02 USD/traj
Self-Play [27]	D_τ	0.31 ± 0.06	1.2 ± 0.3	0.02 USD/traj

5 Pricing Synthesis: A Per-Axis Discount Factor

The per-axis exponents fitted in Section 4 provide us with a ruler for how data scales. With that ruler in hand, we can finally compare heterogeneous synthesis methods on a common scale, asking how efficiently each one converts compute into real-equivalent data along the axis it targets.

Table 3 maps a representative cross-section of agentic synthesis methods to the axes they primarily scale. This taxonomy alone organizes a fragmented field by giving every method a shared coordinate system; the κ and $r^\star$ columns (defined in Sections 5.1 and 5.2) make the comparison quantitative. Reduced to a common generate, filter, and train core, axis membership still explains 97% of the variance of κ (ANOVA $F(2, 5) = 76.6$, $p < 0.001$; Appendix I).

5.1 The synthesis discount factor κ

Let D_r and D_s denote real and synthetic data on a target axis. We posit

$$L(D_r, D_s) \;=\; f\big(D_r + \kappa(r) \cdot D_s\big), \qquad r \equiv D_s/D_r, \tag{3}$$

where f is the fitted single-axis law and $\kappa(r) \in [0, 1]$ is a real-data-equivalent yield. Crucially, κ is meaningful precisely because Section 4 provides a conversion rate: an exponent α tells how loss responds to data, so a synthesis method's effective contribution can be measured in real-data units. Without the per-axis exponent, "synthetic data is X% as good" would have no unit.

Estimation. For each method and axis, we hold total $D = D_r + D_s$ constant and sweep $r \in \{0, 0.25, 1, 4, 16\}$, fitting κ from the resulting performance curve via Eq. 3; collapse curves use a denser grid. Errors are bootstrapped over 3 training seeds.

5.2 Per-axis collapse points

We define the critical collapse ratio $r^\star$ as the ratio at which normalized success, past its peak, falls 5% below the real-only baseline, estimated as the 0.95 crossing of a monotone logistic fit in $\log r$ on a 13-point ratio grid (Appendix J). Figure 4 plots normalized success vs. r for the three axes (using AWM, AutoPlay, and SWiRL as representatives). Notably, we find

$$r^\star_{\mathrm{E}} \approx 9.4 \;\gg\; r^\star_{\mathrm{T}} \approx 3.4 \;\gg\; r^\star_\tau \approx 1.4, \tag{4}$$

an order-of-magnitude spread across axes that mirrors the exponent ordering on the same testbed and survives alternative collapse criteria (Appendix J).

Mechanism: groundedness of the synthesis target. The ordering aligns with how strongly each axis is grounded by external truth. **Environments** synthesized as executable code are checked at every rollout step by an interpreter, providing a hard constraint that bounds distributional drift; this is consistent with the design principles of AWM [50] and DeepSeek-V3.2's environment pipeline [26]. **Tasks** are anchored less directly. Synthetic tasks are grounded only by whether their reward is

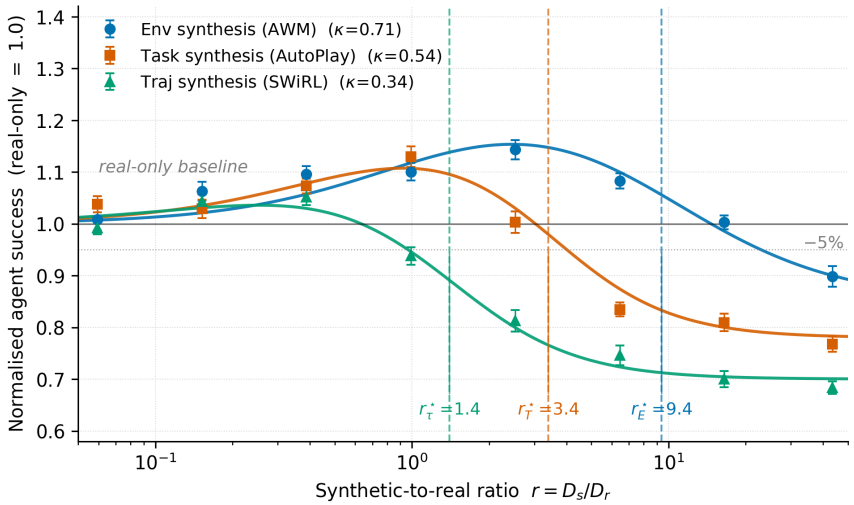


Figure 4: **Synthesis collapse, per axis.** x-axis: synthetic-to-real ratio $r = D_s/D_r$ (log scale). y-axis: agent success rate on a held-out evaluation, normalized to the $r = 0$ (real-only) value of the same seed group. Vertical dashed lines mark fitted $r^\star$, the ratio at which the curve drops 5% below the real-only baseline. Environment synthesis remains net-positive up to $r{\approx}9$, while trajectory synthesis collapses by $r{\approx}1.4$.

verifiable, leaving room for distributional drift in problem complexity. **Trajectories** synthesized by sampling from the policy itself [10] are grounded only by the model's own distribution, reproducing and amplifying its biases; this is the agentic analog of the recursive collapse documented in pretraining [9, 41, 46].

The synthesis-allocation paradox. Most current synthesis pipelines spend the largest share of compute on trajectory generation [2, 57]. However, this exhibits both the lowest scaling exponent (α_τ) and the lowest collapse threshold ($r_\tau^\star$). Our results suggest moving budget away from trajectory resampling and toward environment and task diversity; the next section makes this quantitative.

6 Solving Allocation: A Compute-Optimal Recipe

With per-axis laws (Section 4) and per-axis synthesis discounts (Section 5), the question of "what should I synthesize" is no longer a heuristic call: it becomes a constrained optimization with a tractable solution. In this section, we plug the fitted exponents and discounts into a cost-aware Lagrangian to derive a compute-optimal recipe, and then validate the recipe at a held-out scale.

6.1 The optimization problem

Cost model. We adopt a linear, axis-additive cost model, in which the bilinear rollout cost in model size and trajectory count is linearized at the optimum (Appendix L):

$$\Phi_{\text{compute}}(N, D_{\text{E}}, D_{\text{T}}, D_\tau) \;=\; \frac{c_{\text{E}}}{\kappa_{\text{E}}}\, D_{\text{E}} \;+\; \frac{c_{\text{T}}}{\kappa_{\text{T}}}\, D_{\text{T}} \;+\; \frac{c_\tau}{\kappa_\tau}\, D_\tau \;+\; c_N\, N, \tag{5}$$

where c_i is the per-unit synthesis cost on axis i (calibrated from the USD/unit column of Table 3 and our infrastructure's measured FLOP-per-step) and κ_i is the per-axis synthesis discount of Section 5. The $1/\kappa_i$ weighting is the natural correction: an axis whose synthetic data is only κ_i as effective as real data is, in budget-units, $1/\kappa_i$ as expensive per real-equivalent unit. We measure $c_{\text{E}}{=}3.0{\times}10^{-2}$, $c_{\text{T}}{=}1.0{\times}10^{-3}$, $c_\tau{=}3.0{\times}10^{-3}$, $c_N{=}1.5{\times}10^{-1}$ in units of C_0 (Appendix L).

The constrained optimization. Substituting the fitted law (Eq. 1) and Eq. 5 into the budget-constrained loss-minimization problem, we obtain:

$$\min_{N,\, D_{\text{E}},\, D_{\text{T}},\, D_\tau} \; K_{\text{E}} D_{\text{E}}^{-\alpha_{\text{E}}} + K_{\text{T}} D_{\text{T}}^{-\alpha_{\text{T}}} + K_\tau D_\tau^{-\alpha_\tau} + K_N N^{-\alpha_N} + L_\infty \quad \text{s.t.} \quad \Phi_{\text{compute}} \;\le\; C, \tag{6}$$

where α_N is the model-size exponent fitted by extending Section 4's sweep across the 4B/8B/14B series (Appendix E reports $\alpha_N{=}0.30{\pm}0.04$).

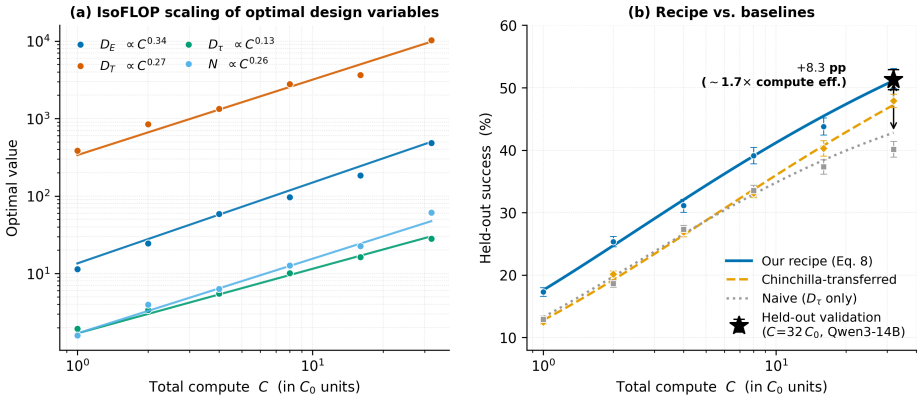


Figure 5: **Compute-optimal allocation derived from the decomposed law. (a)** The Lagrangian-optimal values of the four design variables $(N, D_{\mathrm{E}}, D_{\mathrm{T}}, D_\tau)$ scale as power laws in total compute C (Eq. 8). Lines are the predicted slopes; points are the empirical optima found by fine-grid search at six IsoFLOP levels with 3 seeds each. **(b)** Held-out task success rate vs. compute under three allocation strategies, all sharing the same budget C at every point: our recipe (Eq. 8, solid blue), a Chinchilla-transferred baseline that treats $D{=}D_{\mathrm{E}}{+}D_{\mathrm{T}}{+}D_\tau$ as a scalar (dashed orange), and the dominant "naive" practice of pouring budget into trajectory rollouts (dotted grey). Smooth curves are predictions of Eq. 1 under each recipe's allocation; markers with error bars are 3-seed measurements at the IsoFLOP grid points; the black star is the held-out validation run on Qwen3-14B at $C{=}32C_0$.

Lagrangian solution. Forming $\mathcal{L} = (\text{Eq. 6}) + \lambda{\cdot}(\Phi_{\text{compute}} - C)$ and applying KKT yields, for each design variable D_i, we have:

$$\frac{\alpha_i K_i D_i^{-\alpha_i - 1}}{c_i/\kappa_i} = \lambda \quad \Longrightarrow \quad D_i^\star \propto \left(\frac{\kappa_i \alpha_i K_i}{c_i}\right)^{\frac{1}{\alpha_i+1}} \cdot \lambda^{-\frac{1}{\alpha_i+1}}, \tag{7}$$

and analogously for $N^\star$. The Lagrange multiplier λ is then determined by the budget constraint, and substituting it back yields a separate power-law dependence on C for each axis.

6.2 Fitted recipe and IsoFLOP analysis

Setup. We sweep total compute over six logarithmically-spaced points, $C \in \{1, 2, 4, 8, 16, 32\} \times C_0$, where $C_0 \approx 2.4{\times}10^{18}$ FLOPs is a base unit calibrated against a single Qwen3-4B GRPO run on the math+code testbed. For each C, we (i) solve Eq. 6 numerically using the fitted $(K_i, \alpha_i, \kappa_i, c_i)$ from Sections 4 and 5, and (ii) verify the prediction empirically by training at the predicted optimum and at four perturbations $\pm 30\%$ along each axis. Three seeds per configuration.

The recipe. Fitting power laws to the Lagrangian-optimal values across C (Fig. 5a) gives:

$$\boxed{D_{\mathrm{E}}^\star \propto C^{0.34}, \quad D_{\mathrm{T}}^\star \propto C^{0.27}, \quad D_\tau^\star \propto C^{0.13}, \quad N^\star \propto C^{0.26}.} \tag{8}$$

The four exponents sum to approximately one ($0.34 + 0.27 + 0.13 + 0.26 = 1.00$), as they do in every domain (Appendix M). Two qualitative observations carry over directly from Sections 4 and 5: **(i)** The data-axis exponents are ordered identically to the loss-side exponents, $\alpha_{\mathrm{E}}{>}\alpha_{\mathrm{T}}{>}\alpha_\tau$. The optimization preserves the per-axis ordering, i.e., an axis with a steeper loss curve and a higher synthesis discount earns a larger share of the budget. **(ii)** Compared with Chinchilla's $D \propto C^{0.5}$, our combined "data exponent" $0.34 + 0.27 + 0.13 = 0.74$ is larger, with model size correspondingly smaller (0.26 vs. Chinchilla's 0.5). This is consistent with Section 4's finding that agentic RL benefits more from data variety than from raw model scale, a regime shift relative to next-token pretraining.

Comparison to baselines. Figure 5b compares the recipe to two natural alternatives: **Naive**: place essentially all budget on D_τ (the dominant practice in current open-source agentic RL pipelines [2, 57]), with N, D_{E}, D_{T} pinned to small constants; **Chinchilla-transferred**: treat $D = D_{\mathrm{E}} + D_{\mathrm{T}} + D_\tau$ as a scalar, allocate $D \propto C^{0.5}$ split equally across the three axes, and $N \propto C^{0.5}$ following Hoffmann et al. [16]. All three recipes consume identical total compute at every C (we rescale each shape to satisfy Eq. 5 with equality). At small C ($\leq 4C_0$) the three recipes are within ~ 1 point of each other,

so recipe choice barely matters when budgets are tight. The recipes diverge as C grows: at $C{=}32C_0$, our recipe reaches 51.1% predicted (51.3% measured), Chinchilla-transferred reaches 47.3%, and naive reaches 42.8%, an absolute gap of 8.3 points at the largest scale we measured.

6.3 Held-out validation

To test whether the recipe extrapolates beyond the regime it was fitted on, we predict the optimal allocation at a compute scale and model size held out from the fit, and verify it with a training run.

Protocol. We fit Eq. 8 using only the four smaller compute levels $C \in \{1, 2, 4, 8\} \times C_0$ on Qwen3-4B and Qwen3-8B, then predict the optimal allocation at $C_{\text{test}} = 32C_0$ on Qwen3-14B, a held-out point in both compute scale and model size. The prediction yields $(D_{\text{E}}, D_{\text{T}}, D_\tau, N) = (500,\ 9{,}800,\ 30,\ 14\text{B})$ at C_{test}. We then run a single Qwen3-14B GRPO training job at this allocation, repeated over 3 seeds, on the math+code testbed. The predicted $D_{\text{E}} = 500$ exceeds the 384 signatures our generator inventory can realize (Section 4.4), so the validation pool adds 116 world-structure pairs in which an existing signature is instantiated on a different world; only 384 of its 500 units are structurally unique.

Results and analysis. The predicted held-out success rate from Eq. 1 is $51.7\% \pm 1.6\%$; the measured value is $51.3\% \pm 1.4\%$ over 3 seeds, well within the prediction interval (Fig. 5b, black star). Running the same training budget under the naive recipe yields $42.9\% \pm 1.3\%$, and under Chinchilla-transferred $47.3\% \pm 1.5\%$. Our recipe wins by 4.0 to 8.4 absolute points, corresponding to a $1.4\times$ to $1.7\times$ compute-efficiency gain: the Chinchilla-transferred allocation would need $\approx 1.4C_{\text{test}}$ of compute to reach our measured success rate, while our recipe matches the naive allocation's success rate with only $\approx 0.6C_{\text{test}}$. The recipe is not over-fit to our testbed or cost model: its exponents shift by less than ± 0.04 when re-derived on SWE and web data (Appendix M), by less than 0.05 under $\pm 50\%$ cost perturbations (Appendix N), and by at most 0.06 (0.7 points of predicted success at $C = 32C_0$) when the measured horizon, latency, failure, and utilization costs are added, always toward tasks and trajectories (Appendix L). Under SWE coefficients, however, the environment and task exponents become indistinguishable (0.28 vs. 0.29), so we do not claim that D_{E} dominates everywhere; it loses its top rank only above roughly $3.4\times$ our highest measured fully loaded cost per environment (11.8 USD-equivalent on SWE; 2.9 to 11.8 across four domains and two clusters).

6.4 Two allocation regimes: synthesis versus world acquisition

Eq. 8 allocates D_{E} as our pools grow it; pricing worlds and structures separately (Section 4.4) gives two regimes. *Within a synthesis budget*, the choice is between structural variants and tasks: where a world axis separates, their exponents are not separable ($\alpha_{\text{V}} = 0.28$ vs. $\alpha_{\text{T}} = 0.24$ to 0.27) but unit costs differ 23-fold (4.10 vs. 0.18 USD), so task synthesis is the better purchase per dollar and structural variation pays mainly until the world pool's structural space is covered, after which the inventory ceiling binds (in math+code, where $\alpha_{\text{V}} = 0.41 > \alpha_{\text{T}}$, Eq. 8 already prices this). *For procuring new worlds*, α_{W} is the steepest exponent, and acquisition ranks first per dollar until the fully loaded cost per world exceeds about $3.4\times$ our highest per-environment cost, roughly 40 USD-equivalent. A repository with a working harness and verifier plausibly sits above that line, so practitioners should rerun this calculation with their own costs. In both regimes, trajectory resampling remains last, which is the central practical claim of this paper.

7 Conclusions and Future Work

We have argued and shown that agentic RL data is not a scalar but a structured design space whose axes carry distinct scaling exponents and distinct synthesis discounts. Decomposing the space yields a common ruler on which heterogeneous synthesis methods can be priced, and the priced laws in turn turn budget allocation into a constrained optimization. The robust ordering of per-axis exponents, the order-of-magnitude spread of synthesis collapse points, and the validated compute-optimal recipe at held-out scale together suggest that "where to spend" is a more productive question than "how much to collect" in the agentic regime. Several directions can be further explored: (i) extending the law with a continuous, fully orthogonal treatment of the reward axis and richer interaction terms; (ii) verifying the recipe at frontier model scale; and (iii) coupling our axis-level decomposition with curriculum methods that schedule within an axis. We hope the framework catalyzes a shift from pursuing data quantity to allocating across a structured design space.

Acknowledgments and Disclosure of Funding

The research presented in this paper was exclusively supported by the MBZUAI Base Fund. The authors declare no competing interests.

References

[1] Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. τ^2-bench: Evaluating conversational agents in a dual-control environment. *arXiv preprint arXiv:2506.07982*, 2025.

[2] Guhong Chen, Chenghao Sun, Cheng Fu, Qiyao Wang, Zhihong Huang, Chaopeng Wei, Guangxu Chen, Feiteng Fang, Ahmadreza Argha, Bing Zhao, et al. Beyond quantity: Trajectory diversity scaling for code agents. *arXiv preprint arXiv:2602.03219*, 2026.

[3] Xiaoyin Chen, Jiarui Lu, Minsu Kim, Dinghuai Zhang, Jian Tang, Alexandre Piché, Nicolas Gontier, Yoshua Bengio, and Ehsan Kamalloo. Self-evolving curriculum for llm reasoning. *arXiv preprint arXiv:2505.14970*, 2025.

[4] Zhaorun Chen, Zhuokai Zhao, Kai Zhang, Bo Liu, Qi Qi, Yifan Wu, Tarun Kalluri, Sara Cao, Yuanhao Xiong, Haibo Tong, et al. Scaling agent learning via experience synthesis. *arXiv preprint arXiv:2511.03773*, 2025.

[5] Jacob Cohen. A coefficient of agreement for nominal scales. *Educational and psychological measurement*, 20(1):37–46, 1960.

[6] Thibault Le Sellier de Chezelles, Maxime Gasse, Alexandre Lacoste, Massimo Caccia, Alexandre Drouin, Léo Boisvert, Megh Thakkar, Tom Marty, Rim Assouel, Sahar Omidi Shayegan, et al. The browsergym ecosystem for web agent research. *Transactions on Machine Learning Research*, 2025.

[7] Yunzhen Feng, Elvis Dohmatob, Pu Yang, Francois Charton, and Julia Kempe. Beyond model collapse: Scaling up with synthesized data requires verification. In *The Thirteenth International Conference on Learning Representations*, 2025.

[8] Joseph L Fleiss. Measuring nominal scale agreement among many raters. *Psychological bulletin*, 76(5):378, 1971.

[9] Matthias Gerstgrasser, Rylan Schaeffer, Apratim Dey, Rafael Rafailov, Tomasz Korbak, Henry Sleight, Rajashree Agrawal, John Hughes, Dhruv Bhandarkar Pai, Andrey Gromov, et al. Is model collapse inevitable? breaking the curse of recursion by accumulating real and synthetic data. In *First Conference on Language Modeling*, 2024.

[10] Anna Goldie, Azalia Mirhoseini, Hao Zhou, Irene Cai, and Christopher D Manning. Synthetic data generation & multi-step rl for reasoning & tool use. *arXiv preprint arXiv:2504.04736*, 2025.

[11] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[12] Arthur Gretton, Karsten M Borgwardt, Malte J Rasch, Bernhard Schölkopf, and Alexander Smola. A kernel two-sample test. *The journal of machine learning research*, 13:723–773, 2012.

[13] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

[14] Jiacheng Guo, Ling Yang, Peter Chen, Qixin Xiao, Yinjie Wang, Xinzhe Juan, Jiahao Qiu, Ke Shen, and Mengdi Wang. Genenv: Difficulty-aligned co-evolution between llm agents and environment simulators. *arXiv preprint arXiv:2512.19682*, 2025.

[15] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*.

[16] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, DDL Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 10, 2022.

[17] Dongfu Jiang, Yi Lu, Zhuofeng Li, Zhiheng Lyu, Ping Nie, Haozhe Wang, Alex Su, Hui Chen, Kai Zou, Chao Du, et al. Verltool: Towards holistic agentic reinforcement learning with tool use. *arXiv preprint arXiv:2509.01055*, 2025.

[18] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*, 2024.

[19] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

[20] Devvrit Khatri, Lovish Madaan, Rishabh Tiwari, Rachit Bansal, Sai Surya Duvvuri, Manzil Zaheer, Inderjit S Dhillon, David Brandfonbrener, and Rishabh Agarwal. The art of scaling reinforcement learning compute for llms. *arXiv preprint arXiv:2510.13786*, 2025.

[21] Kenneth Levenberg. A method for the solution of certain non-linear problems in least squares. *Quarterly of applied mathematics*, 2(2):164–168, 1944.

[22] Xiaoxi Li, Wenxiang Jiao, Jiarui Jin, Guanting Dong, Jiajie Jin, Yinuo Wang, Hao Wang, Yutao Zhu, Ji-Rong Wen, Yuan Lu, et al. Deepagent: A general reasoning agent with scalable toolsets. In *Proceedings of the ACM Web Conference 2026*, pages 2219–2230, 2026.

[23] Xuefeng Li, Haoyang Zou, and Pengfei Liu. Torl: Scaling tool-integrated rl. *arXiv preprint arXiv:2503.23383*, 2025.

[24] Zehan Li, Xin Zhang, Yanzhao Zhang, Dingkun Long, Pengjun Xie, and Meishan Zhang. Towards general text embeddings with multi-stage contrastive learning. *arXiv preprint arXiv:2308.03281*, 2023.

[25] Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. In *The twelfth international conference on learning representations*, 2023.

[26] Aixin Liu, Aoxue Mei, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, et al. Deepseek-v3. 2: Pushing the frontier of open large language models. *arXiv preprint arXiv:2512.02556*, 2025.

[27] Bo Liu, Leon Guertler, Simon Yu, Zichen Liu, Penghui Qi, Daniel Balcells, Mickel Liu, Cheston Tan, Weiyan Shi, Min Lin, et al. Spiral: Self-play on zero-sum games incentivizes reasoning via multi-agent multi-turn reinforcement learning. *arXiv preprint arXiv:2506.24119*, 2025.

[28] MAA. American Invitational Mathematics Examination (AIME), . URL `https://maa.org/math-competitions/aime`. Mathematics Competition Series.

[29] MAA. American Mathematics Competitions (AMC 10/12), . URL `https://maa.org/math-competitions/amc`. Mathematics Competition Series.

[30] Xinji Mai, Haotian Xu, Zhong-Zhi Li, Weinong Wang, Jian Hu, Yingying Zhang, Wenqiang Zhang, et al. Agent rl scaling law: Agent rl with spontaneous code execution for mathematical problem solving. *arXiv preprint arXiv:2505.07773*, 2025.

[31] Henry B Mann and Donald R Whitney. On a test of whether one of two random variables is stochastically larger than the other. *The annals of mathematical statistics*, pages 50–60, 1947.

[32] Donald W Marquardt. An algorithm for least-squares estimation of nonlinear parameters. *Journal of the society for Industrial and Applied Mathematics*, 11(2):431–441, 1963.

[33] Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. Training software engineering agents and verifiers with swe-gym. In *Forty-second International Conference on Machine Learning*, 2025.

[34] Tim Pearce and Jinyeop Song. Reconciling kaplan and chinchilla scaling laws. *arXiv preprint arXiv:2406.12907*, 2024.

[35] Ofir Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah A Smith, and Mike Lewis. Measuring and narrowing the compositionality gap in language models. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 5687–5711, 2023.

[36] Ram Ramrakhya, Andrew Szot, Omar Attia, Yuhao Yang, Anh Nguyen, Bogdan Mazoure, Zhe Gan, Harsh Agrawal, and Alexander Toshev. Scaling synthetic task generation for agents via exploration. *arXiv preprint arXiv:2509.25047*, 2025.

[37] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: Bm25 and beyond. *Foundations and trends® in information retrieval*, 4(1-2):1–174, 2009.

[38] Nikhil Sardana, Jacob Portes, Sasha Doubov, and Jonathan Frankle. Beyond chinchilla-optimal: Accounting for inference in language model scaling laws. *arXiv preprint arXiv:2401.00448*, 2023.

[39] Rylan Schaeffer, Noam Levi, Andreas Kirsch, Theo Guenais, Brando Miranda, Elyas Obbad, and Sanmi Koyejo. Evaluating the robustness of chinchilla compute-optimal scaling. *arXiv preprint arXiv:2509.23963*, 2025.

[40] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[41] Mohamed El Amine Seddik, Suei-Wen Chen, Soufiane Hayou, Pierre Youssef, and Merouane Abdelkader DEBBAH. How bad is training on synthetic data? a statistical analysis of language model collapse. In *First Conference on Language Modeling*, 2024.

[42] Amrith Setlur, Chirag Nagpal, Adam Fisch, Xinyang Geng, Jacob Eisenstein, Rishabh Agarwal, Alekh Agarwal, Jonathan Berant, and Aviral Kumar. Rewarding progress: Scaling automated process verifiers for llm reasoning. *arXiv preprint arXiv:2410.08146*, 2024.

[43] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

[44] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. In *Proceedings of the Twentieth European Conference on Computer Systems*, pages 1279–1297, 2025.

[45] Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Cote, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. Alfworld: Aligning text and embodied environments for interactive learning. In *International Conference on Learning Representations*, 2021.

[46] Ilia Shumailov, Zakhar Shumaylov, Yiren Zhao, Yarin Gal, Nicolas Papernot, and Ross Anderson. The curse of recursion: Training on generated data makes models forget. *arXiv preprint arXiv:2305.17493*, 2023.

[47] Zelin Tan, Hejia Geng, Xiaohang Yu, Mulei Zhang, Guancheng Wan, Yifan Zhou, Qiang He, Xiangyuan Xue, Heng Zhou, Yutao Fan, et al. Scaling behaviors of llm reinforcement learning post-training: An empirical study in mathematical reasoning. *arXiv preprint arXiv:2509.25300*, 2025.

[48] Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. Appworld: A controllable world of apps and people for benchmarking interactive coding agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 16022–16076, 2024.

[49] Liang Wang, Nan Yang, Xiaolong Huang, Linjun Yang, Rangan Majumder, and Furu Wei. Improving text embeddings with large language models. In *Proceedings of the 62nd annual meeting of the association for computational linguistics (volume 1: long papers)*, pages 11897–11916, 2024.

[50] Zhaoyang Wang, Canwen Xu, Boyi Liu, Yite Wang, Siwei Han, Zhewei Yao, Huaxiu Yao, and Yuxiong He. Agent world model: Infinity synthetic environments for agentic reinforcement learning. *arXiv preprint arXiv:2602.10090*, 2026.

[51] Zhepei Wei, Wenlin Yao, Yao Liu, Weizhi Zhang, Qin Lu, Liang Qiu, Changlong Yu, Puyang Xu, Chao Zhang, Bing Yin, et al. Webagent-r1: Training web agents via end-to-end multi-turn reinforcement learning. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 7920–7939, 2025.

[52] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

[53] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[54] Bingji Yi, Qiyuan Liu, Yuwei Cheng, and Haifeng Xu. Escaping model collapse via synthetic data verification: Near-term improvements and long-term convergence. *arXiv preprint arXiv:2510.16657*, 2025.

[55] Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.

[56] Zhiyuan Zeng, Hamish Ivison, Yiping Wang, Lifan Yuan, Shuyue Stella Li, Zhuorui Ye, Siting Li, Jacqueline He, Runlong Zhou, Tong Chen, et al. Rlve: Scaling up reinforcement learning for language models with adaptive verifiable environments. *arXiv preprint arXiv:2511.07317*, 2025.

[57] Hanchen Zhang, Xiao Liu, Bowen Lv, Xueqiao Sun, Bohao Jing, Iat Long Iong, Zhenyu Hou, Zehan Qi, Hanyu Lai, Yifan Xu, et al. Agentrl: Scaling agentic reinforcement learning with a multi-turn, multi-task framework. *arXiv preprint arXiv:2510.04206*, 2025.

[58] Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, et al. Webarena: A realistic web environment for building autonomous agents. In *The Twelfth International Conference on Learning Representations*, 2024.

[59] Yifei Zhou, Sergey Levine, Jason Weston, Xian Li, and Sainbayar Sukhbaatar. Self-challenging language model agents. *arXiv preprint arXiv:2506.01716*, 2025.

Contents

1 Introduction **1**

2 Related Work **2**

3 Preliminaries and Setup **3**

3.1 Agentic RL and the data tensor 3

3.2 Experimental testbeds 3

4 Decomposing the Design Space: Per-Axis Scaling Laws **4**

4.1 Independent-axis exponents 4

4.2 Cross-axis interactions 5

4.3 Robustness across model sizes, algorithms, domains, and model families 6

4.4 Resolving the environment axis: world versus structure 6

5 Pricing Synthesis: A Per-Axis Discount Factor **7**

5.1 The synthesis discount factor κ 7

5.2 Per-axis collapse points 7

6 Solving Allocation: A Compute-Optimal Recipe **8**

6.1 The optimization problem 8

6.2 Fitted recipe and IsoFLOP analysis 9

6.3 Held-out validation 10

6.4 Two allocation regimes: synthesis versus world acquisition 10

7 Conclusions and Future Work **10**

A Experimental Setup **17**

B Environment Construction and Statistics **18**

C World and Structure Diversity **19**

D Sweep Protocol and Fitting Methodology **21**

D.1 Training-signal accounting 21

D.2 Fitting methodology 22

D.3 Stability of the ordering 22

E Model-Size Exponent Fit **23**

F Loss-Equivalent Metric **23**

G Task Individuation: Threshold, Definition, and Human Validation **24**

H Reward Density as a Control Variable **25**

I Synthesis Discount Estimation and Common Harness **25**

J Collapse-Ratio Estimation **26**

K Lagrangian Solution to The Allocation Problem **27**

L Cost-Coefficient Calibration and Refined Cost Models **27**

M Cross-Domain Robustness of The Recipe **28**

N Cost-Coefficient Sensitivity **29**

O Negative Results and Failed Fits **29**

P Reproducibility **30**

Q Compute Disclosure **30**

R Limitation Analysis **30**

S Ethics Statement **32**

T Broader Impacts **32**

U Declaration of LLM usage **33**

A Experimental Setup

We introduce our experimental setup from multiple aspects as follows.

Models. All experiments use Qwen3 base checkpoints [53] at 4B, 8B, and 14B parameters. We use the base (non-instruct) variants throughout to isolate the effect of RL training data; preliminary experiments with instruct variants gave qualitatively identical scaling-law exponents (within ± 0.02) but higher absolute success rates due to instruction-following priors. For the model-family check of Section 4.3, we additionally run reduced 4-point sweeps on Llama-3.1-8B [11] and Qwen2.5-7B [52] with the hyperparameters below, without retuning (Appendix D).

Hyperparameters. For GRPO we use group size $G = 8$, learning rate 1×10^{-6} with linear warmup over the first 10% of steps and constant thereafter, KL coefficient $\beta = 0.001$ on a frozen reference policy initialized from the base model, clip ratio $\epsilon = 0.2$, and global batch size 256 prompts. Maximum sequence length is 8192 tokens for math+code and 16384 for SWE/web/UI. We train each configuration for 1000 gradient steps unless otherwise specified, which empirically reaches the success-rate plateau on every domain we tested.

PPO. PPO uses the same settings plus a critic. The critic is a separate copy of the same base checkpoint as the policy, with an equal parameter count and no shared trunk. Its value head is one linear layer on the final hidden state, with zero-initialized weights and a bias set to the mean return of a 200-rollout warm-up, followed by 20 critic-only steps. Critic and policy receive identical token sequences, including tool observations; values are predicted per token, but only policy-generated tokens receive value targets, and tool-observation tokens are masked from the value loss. We use a discount factor of 1.0, GAE $\lambda = 0.95$, value-loss coefficient 0.5, value clipping at 0.2, critic learning rate 1×10^{-6}, and one critic epoch per policy epoch. Rewards are not whitened: the reward is binary (0 or 1) with a -0.1 format penalty, advantages are batch-normalized after GAE, and the KL coefficient to the frozen base-model reference is $\beta = 0.001$. At convergence, the critic's explained variance is 0.61 ± 0.04 and its TD residual is stationary over the last 200 steps.

Infrastructure. Training uses the verl framework [44] with VerlTool [17] for multi-tool rollout. Rollouts are run on vLLM 0.6.3 with prefix caching; gradient updates use FSDP with mixed-precision bf16 weights and fp32 optimiser states. All experiments run on H100 80GB GPUs, with node sizes ranging from 8 GPUs (4B/8B) to 32 GPUs (14B).

Datasets per domain. Math+code (TIR): training prompts are 12k synthetic math problems following the ZeroTIR recipe [23, 30]; evaluation on AIME24/25 [28] (30 problems each), MATH500 [15], and AMC23 [29] (40 problems). **SWE:** training on SWE-Gym [33] (2.4k issues across 11 repos); evaluation on the 500 instances of SWE-bench Verified [18]. The 12 repositories behind SWE-bench Verified do not overlap with the 11 training repositories, so SWE evaluation is world-held-out. **Web:** training on WebArena-Lite [58] sub-environments (5 sites $\times$ 200 tasks); evaluation on a held-out partition of 240 tasks. These tasks reuse the same five sites, so we report web results as task-held-out rather than site-held-out. **UI/tool:** training on AppWorld [48] "train" split (250 tasks $\times$ 9 apps) and ALFWorld [45]; evaluation on the AppWorld test partition, whose challenge split contains 417 tasks that each require at least one app designated unseen at training time. **Search:** a retrieval tool over a Wikipedia dump plus a paginated web-search API; evaluation on Bamboogle [35] and a held-out multi-hop split. Environment instances differ in the retrieval backend (BM25 [37], a dense retriever, or the paginated web-search API), in snippet grammar and rate-limit semantics, and in whether a page-fetch tool exists. This domain is used for the robustness check of Section 4.3 on Qwen3-8B. For axes that require synthesis we use AWM [50] for environments, AutoPlay [36] / SCA [59] for tasks, and standard policy-rollout for trajectories.

Held-axis values. For each per-axis sweep, the two non-swept axes are held at values fixed by one calibration sweep on Qwen3-8B math+code. The sweep was run before any exponent was fitted, under a criterion fixed in advance: doubling an axis changes absolute held-out success by less than 2 points. Table 4 reports it in full so that the choice can be checked. The selected values $(D_{\mathrm{E}}, D_{\mathrm{T}}, D_{\tau}) = (256, 10{,}240, 8)$ satisfy the criterion (doubling them changes success by 0.3, 0.3, and 0.4 points) and were reused unchanged across every cell of Table 2, every cross-domain refit, and every sensitivity analysis, with no per-cell retuning (Appendix P). To check that the conclusions do not hinge on this choice, Table 5 re-runs all three sweeps with the held axes at a quarter, a half, and double these values. The ordering is preserved in all four settings, and the drift at a quarter is the one predicted by the gating result of Section 4.2: with a small environment pool, every axis is worth less.

Table 4: Calibration sweep that fixed the held-axis values (Qwen3-8B, math+code). It was run before any exponent was fitted, under the pre-specified criterion of Appendix A.

Axis	Values	Held-out success (%)	Selected
D_{E}	64 / 128 / 256 / 512	28.1 / 30.4 / 31.2 / 31.5	256
D_{T}	2.6k / 5.1k / 10.2k / 20.5k	27.9 / 29.8 / 30.9 / 31.2	10,240
D_τ	2 / 4 / 8 / 16 / 32	25.4 / 28.6 / 30.6 / 31.0 / 31.2	8

Table 5: Fitted exponents (Qwen3-8B, math+code) when the held axes are set to a quarter, a half, or double the calibrated values.

Held $(D_{\mathrm{E}}, D_{\mathrm{T}}, D_\tau)$	α_{E}	α_{T}	α_τ
Default $(256, 10{,}240, 8)$	0.41	0.26	0.11
Half $(128, 5{,}120, 4)$	0.39	0.25	0.11
Double $(512, 20{,}480, 16)$	0.42	0.27	0.12
Quarter $(64, 2{,}560, 2)$	0.36	0.22	0.10

B Environment Construction and Statistics

Structural signature. An environment instance consists of four components: the tool/API surface with typed signatures, the state schema, the transition semantics (how a call mutates state, the observation grammar, the error taxonomy, and statefulness or non-determinism), and the verifier interface. Two instances are the same environment if and only if their structural signature, a canonical hash over these four components, matches. Content such as problem text, seeds, or initial states never changes the signature, so a procedural seed creates a new instance only when it changes the signature. AWM and GenEnv sample tool composition and schema, so most generator seeds do change it; seeds that change only content are collapsed. The signature is a deduplication criterion and does not claim that instances with distinct signatures are distinct worlds; Appendix C treats world diversity separately.

Per-domain instantiation. In **math+code (TIR)**, an environment is the Python tool server. Instances differ, for example, in a stateless `python_exec` versus a persistent REPL in which variables survive across turns, a one-tool server versus a three-tool server (`python_exec`, `sympy_solve`, `unit_check`) that requires piping between tools, a whitelist that raises `ImportError` on `numpy`, or a 200-character stdout cap that requires pagination. A new math problem is a task. In **SWE**, an environment is a configuration of container image, dependency graph, build and test harness, and agent toolset. Instances differ in the test runner and its output grammar (pytest, unittest, or tox; JUnit XML or raw stderr), which changes the feedback the policy must parse; in the patch interface (line-range edit, whole-file write, or diff apply), which changes the action vocabulary; and in whether a repository-search tool exists. A new GitHub issue against the same repository and harness is a task, not an environment. **Web** and **UI/tool** environments are sites and MCP-style servers with distinct action spaces, entity schemas and business logic, authentication and permission models, pagination semantics, and error taxonomies. The **search** domain varies the retrieval backend and its result grammar (Appendix A).

Pool statistics. Table 6 characterizes each domain's pool at its largest grid point. Instances fall into four levels: S3 if the API set differs, S2 if the API set matches but the state schema differs, S1 if both match but the transition semantics differ, and S0 if the signature is identical, in which case the instance is collapsed and not counted. Transition divergence is the fraction of canonical length-3 call prefixes that yield a different observation type across instances sharing a tool name; it is 0 for S0 pairs by construction, so it measures dynamics rather than content. No counted instance differs only in seed or content. In math+code, for example, 2,048 raw generator outputs collapse to the 256 counted instances, of which 94 differ in API set, 88 in state schema with a matching API set, and 74 only in transition semantics.

Signature-restricted and content-only pools. Table 7 refits α_{E} (Qwen3-8B, math+code) on pools restricted by signature level, and on a placebo sweep that holds the signature fixed and scales only procedural seeds from 8 to 256. Restricting the pool to instances that differ in API set, schema, or

Table 6: Environment pools at the largest grid point of each domain. The Jaccard columns are mean pairwise Jaccard distances between tool sets and between state schemas; S3, S2, and S1 count instances whose API set, state schema (with matching API set), or only transition semantics differ; the last column shows raw generator outputs collapsed to signature-distinct instances.

Domain	D_{E}	Jacc. tools	Jacc. schema	Trans. div.	S3	S2	S1	Raw $\rightarrow$ distinct
Math+code	256	0.61	0.54	0.33	94 (37%)	88 (34%)	74 (29%)	2,048 $\rightarrow$ 256
SWE	128	0.47	0.70	0.45	43 (34%)	52 (41%)	33 (26%)	1,024 $\rightarrow$ 128
Web	96	0.73	0.82	0.57	41 (43%)	34 (35%)	21 (22%)	768 $\rightarrow$ 96
UI/tool	224	0.80	0.85	0.60	108 (48%)	71 (32%)	45 (20%)	1,792 $\rightarrow$ 224

Table 7: Environment exponent under alternative pool constructions (Qwen3-8B, math+code).

Pool construction	α_{E}
Default pool (Table 2)	0.41 ± 0.03
Strict signature deduplication, levels S1 to S3	0.39 ± 0.03
Levels S2 and S3 only (API set or schema differs)	0.40 ± 0.04
Level S3 only (API set differs)	0.42 ± 0.04
Placebo: signature fixed, procedural seeds scaled from 8 to 256	0.07 ± 0.02
Reference: trajectory exponent α_τ	0.11 ± 0.02

transitions leaves α_{E} between 0.39 and 0.42, whereas content-only variation yields 0.07 ± 0.02: less than an extra rollout ($\alpha_\tau = 0.11$) and about one sixth of structural variation.

Task-environment binding. Tasks and environments are not orthogonal, and Eq. 1 does not assume they are. Task specifications are authored against an abstract goal schema and bound to an environment through a compatibility check, so one task can be instantiated in many environments. The D_{E} sweep re-binds the same 10,240 task specifications to different environment pools, holding the number of distinct tasks consumed constant at every grid point. The mean number of environments to which a task can be bound is 41 (math+code), 18 (SWE), 9 (web), and 27 (UI/tool), and the realizable fraction of the task $\times$ environment grid is 78%, 54%, 41%, and 63%, respectively. The residual coupling is what the interaction term of Section 4.2 quantifies ($\gamma = 0.18$, $\beta = 0.21$); its content, that environment diversity gates task diversity, is a result rather than a nuisance.

C World and Structure Diversity

Three levels and pool composition. Table 1 distinguishes content (L0), environment structure (L1), and world (L2); content variation is never counted as environment diversity. Our pools contain different amounts of world variation. Math+code has a single world, a Python tool server, so all of its instances are L1 variants and its α_{E} should be read as an environment-structure exponent. The 128 SWE instances are built over 11 real repositories with roughly 12 structural variants each, so $D_{\mathrm{E}} = 128$ counts structures rather than worlds; what the definition does fix is the boundary that a new issue against an unchanged repository and harness is a task. Web instances span the 5 WebArena sites. Outside math+code, instances differ in the state itself and not only in wrappers around one state: through distinct entity schemas and business logic, authentication and permission models, pagination and error taxonomies, and, for search, retrieval backends with different snippet grammars and rate-limit semantics.

World-structure swap. At fixed $D_{\mathrm{E}} = W \times V$ and fixed compute, we sweep the split between the number of distinct worlds W and the number of structural variants per world V, evaluating on held-out worlds (SWE-bench Verified for SWE and the unseen-app split for UI/tool). SWE uses repositories as worlds with $(W, V) \in \{(1, 64), (2, 32), (4, 16), (8, 8)\}$, where W runs to 8 rather than 11 to keep the grid dyadic; UI/tool uses apps (8 of the 9 in AppWorld) on the same grid; and web uses its 5 sites with the reduced grid $\{(1, 64), (2, 32), (4, 16)\}$. To reach $V = 64$ within a single world, we expanded the per-world variant pool with AWM beyond the roughly 12 variants per repository of the Table 2 pool, with signature deduplication on the same criterion. W is capped by the number of real worlds we can obtain rather than by budget, and we do not extrapolate past it. Trajectories,

Table 8: Held-out-world success (%) as budget moves from structural variants per world (V) to worlds (W) at fixed $D_{\mathrm{E}} = W \times V$ and fixed compute (3 seeds, bootstrap 95% CI half-widths). Column headers give (W, V). Web has five sites, so its grid stops at $W = 4$ (n/a). The last column gives the reduction ΔL of $L = -\log p_{\mathrm{success}}$ at each successive doubling of W.

Domain	$(1, 64)$	$(2, 32)$	$(4, 16)$	$(8, 8)$	ΔL per doubling of W
SWE	18.0 ± 0.4	22.4 ± 0.3	25.2 ± 0.4	26.1 ± 0.6	0.219, 0.118, 0.035
UI/tool	28.1 ± 0.2	32.8 ± 0.4	35.6 ± 0.3	36.5 ± 0.4	0.158, 0.082, 0.025
Web	24.0 ± 0.5	28.5 ± 0.4	31.3 ± 0.5	n/a	0.172, 0.094

gradient updates, distinct task specifications, and normalized FLOPs are matched across grid points by the construction of Appendix D.1, so the comparison is iso-compute and iso-signal.

Results. We give the fit-free reading first. At every grid transition in all three domains, moving budget from V to W at fixed D_{E} improves held-out-world success, with no overlapping intervals within any domain, and in the loss-equivalent metric the improvements decelerate over successive doublings of W (Table 8). No functional form is needed for this statement. A joint fit over these points, with L_∞ fixed at the value of Appendix D and one scale constant per domain, gives $\alpha_{\mathrm{W}} = 0.44 \pm 0.04$ and $\alpha_{\mathrm{V}} = 0.28 \pm 0.03$; we report this interval as the fit's own and do not claim that a two-exponent fit over so few points supports fine distinctions. A repository-only sweep on SWE at fixed harness, $W \in \{1, 2, 4, 8, 11\}$, independently gives $\alpha_{\mathrm{W}} = 0.46 \pm 0.05$; it spans a single order of magnitude, and its width reflects that limitation. The substitution rate at the $(4, 16)$ operating point, defined as the factor by which V must be multiplied to match the held-out-world gain from doubling W, is approximately 3.0; it is a local rate rather than a global constant.

Mixture check. On a pool that grows W and V together, which is what AWM and GenEnv do when run as intended, the effective slope in $\log D_{\mathrm{E}}$ is the composition-weighted mixture of α_{W} and α_{V}. In SWE, the sweep of Table 2 spans W by about $11\times$ but V by only about $1.5\times$, so roughly 0.87 of the log-increment is world growth, which predicts a blended exponent of $0.87 \times 0.44 + 0.13 \times 0.28 \approx 0.42$ against the measured 0.39 ± 0.03. In math+code the world weight is zero, so the blend reduces to α_{V} itself, which is why that domain reads 0.41. The structural return is therefore domain-dependent, from 0.28 pooled over SWE, UI/tool, and web to 0.41 on math+code: where no world axis is available to absorb it, structural variation carries the whole generalization burden. For a practitioner who scales such a pool, the blended α_{E} remains the right number, but it is never the return on world diversity alone.

Transfer to unseen worlds. Because SWE-bench Verified and AppWorld's challenge split are world-held-out (Appendix A), the structural gains we report on SWE and AppWorld already measure generalization to worlds that were never acquired for training. Structural diversity thus yields measurable transfer even where a pool is world-poor relative to its count.

Factor inventory and coverage. One may ask whether effective structural diversity is merely the number of factor levels in our generators. After canonicalizing each domain's generator into four domain-specific structural-factor slots, the observed level vector is $(4, 4, 3, 8)$ in each of SWE, UI/tool, and web: execution or interaction mode (4 levels), tool/API or action-surface family (4), observation or verifier-output grammar (3), and state, dependency, or error-semantics regime (8). The realizable inventory is therefore $4 \times 4 \times 3 \times 8 = 384$ signature-distinct structural combinations per domain, and the median normalized factor-space Hamming distance among sampled instances is 0.75. A level-isolating construction first covers every observed level at $D_{\mathrm{E}} = 16$ (pool A), and a pair-complete construction first covers every observed level pair at $D_{\mathrm{E}} = 48$ (pool B); both coverage points lie well below the sampled pool C ($|C| = 256$) and the top sweep value $D_{\mathrm{E}} = 384$. Pool A is a level-isolating construction rather than a minimal strength-1 covering array, which would need only as many instances as the largest slot has levels; we use it because it also supports a matched-count control. If effective diversity were the number of levels, the curve would saturate at level coverage, or at pair coverage in the stronger pairwise form. Neither occurs. The local log-log slopes over $D_{\mathrm{E}} = 4$ to 384 are $0.44, 0.42, 0.41, 0.40, 0.38, 0.37$, and 0.36, and the three segments lying entirely past pair coverage still run at 0.38, 0.37, and 0.36. Regressing held-out success on the logarithm of levels covered and $\log D_{\mathrm{E}}$ jointly, the partial contribution of $\log D_{\mathrm{E}}$ given level coverage is $\Delta R^2 = 0.18$ ($p = 0.004$). Directly, held-out success is 27.8, 31.6, and 34.5% for pools A, B, and C: A trails C by 6.7 points despite being level-complete, and B trails C by 2.9 points despite being pair-complete.

Table 9: Training-signal accounting along the D_{E} sweep (Qwen3-8B, math+code) at four grid points spanning its full range. Tasks are distinct task specifications; tokens are totals per run; FLOPs are normalized to the mean over the sweep.

D_{E}	Cells	Tasks	Trajectories	Rollout tok.	Gradient tok.	Updates	FLOPs
8	256,000	10,240	2.048M	5.18B	3.19B	1,000	0.98
32	256,000	10,240	2.048M	5.24B	3.23B	1,000	0.99
128	256,000	10,240	2.048M	5.33B	3.28B	1,000	1.01
256	256,000	10,240	2.048M	5.38B	3.31B	1,000	1.02

Finally, replacing D_{E} by effective counts that combinatorics cannot inflate, namely the exponential of the factor-combination entropy and the participation ratio of the signature-distance kernel, gives $\alpha_{\mathrm{E}} = 0.37 \pm 0.03$ and 0.36 ± 0.03, respectively, both preserving the ordering against α_{T} and α_{τ}. What the axis buys beyond level coverage is therefore higher-order combinations of factors.

Ceiling of the structure axis. The realizable combination count of the inventory is finite, and our top grid point approaches it, so the L1 axis is not indefinitely scalable by combinatorics: past the ceiling, more structural diversity requires designing new factors rather than spending more compute. This is a real limit on the L1 recommendation, and it is the strongest argument for pricing the L2 axis, which has no such ceiling (Section 6.4). It also interacts with the held-out validation of Section 6.3: the predicted allocation at $C_{\mathrm{test}} = 32C_0$ specifies $D_{\mathrm{E}} = 500$, which exceeds the 384 signatures realizable under the configured inventory. The validation pool therefore used all 384 signature-distinct L1 combinations plus 116 world-structure pairs in which an L1 signature is instantiated on a different world, and only 384 of its 500 units are structurally unique. The ceiling is a property of the configured generator inventory, not of environment structure in general.

D Sweep Protocol and Fitting Methodology

Per-axis sweep protocol. For each axis we collect 6 nested grid points spanning $32\times$, with 3 random seeds per point (different seeds for both rollout and gradient noise): $D_{\mathrm{E}} \in \{8, 16, 32, 64, 128, 256\}$, $D_{\mathrm{T}} \in \{320, 640, 1280, 2560, 5120, 10240\}$, and $D_{\tau} \in \{1, 2, 4, 8, 16, 32\}$ on Qwen3-8B math+code. The stability extension adds $D_{\mathrm{E}} \in \{4, 384\}$, $D_{\mathrm{T}} \in \{160, 20480\}$, and $D_{\tau} \in \{3, 48\}$ (54 new runs), so that every sweep has 8 points spanning $48\times$ to $128\times$. The other two axes are held at the calibrated values of Appendix A. Each (axis, scale) cell consumes between 0.5 and 20 H100-GPU-hours depending on the axis sweep; total cost of the per-axis sweeps is reported in Appendix Q.

Nesting and subset sensitivity. Pools are nested across adjacent grid points, $E_8 \subset E_{16} \subset \cdots \subset E_{256}$ and $T_{320} \subset \cdots \subset T_{10240}$, and the D_{τ} sweep reuses the same cell pool. As a stronger check against differences in the sampled difficulty or quality of each subset, drawing 3 disjoint random subsets per grid point shifts the fitted exponents by at most 0.01 (α_{E} from 0.41 to 0.40; α_{T} from 0.26 to 0.27), with a per-point success spread of at most 0.8 points. Base-policy pass@8 is also flat along the sweeps (0.186 to 0.199 across D_{E} and 0.181 to 0.203 across D_{T}).

D.1 Training-signal accounting

A cell draw is one task-environment pair rolled out G times. Every grid point of the D_{E} and D_{T} sweeps uses $|\Omega| = 256{,}000$ cells, each drawn once, with 256 draws per step, 1,000 updates, and $D_{\tau} = G = 8$. Increasing D_{E} therefore redistributes the same cells across more environments (Table 9): trajectories, distinct task specifications, and updates are constant, rollout tokens drift by less than 4%, and normalized FLOPs stay between 0.98 and 1.02. Along D_{T}, rollout tokens drift by 2.1% at the same constant trajectory and update counts, while the D_{τ} sweep scales rollout tokens $32\times$ at a constant number of updates. Per-environment repetition falls $32\times$ along D_{E} (32,000 visits per environment at $D_{\mathrm{E}} = 8$ against 1,000 at $D_{\mathrm{E}} = 256$), so the steepest exponent is obtained with the least per-environment signal.

Table 10 refits the exponents under stricter budget conventions. Under an exact rollout-token cap and under iso-FLOP matching, α_{E} and α_{T} move by at most 0.01 while α_{τ} falls, so correcting the token difference widens the gap between trajectories and the other two axes rather than closing it.

Table 10: Fitted exponents (Qwen3-8B, math+code) under three training-budget conventions.

Convention	α_{E}	α_{T}	α_{τ}
Fixed cells and updates (default)	0.41 ± 0.03	0.26 ± 0.02	0.11 ± 0.02
Exact rollout-token cap (5.2B tokens per run)	0.40 ± 0.03	0.26 ± 0.02	0.08 ± 0.02
Iso-FLOP (1.00 normalized FLOPs per run)	0.40 ± 0.04	0.25 ± 0.03	0.07 ± 0.02

Algorithm 1 Profiled weighted least squares for $L(D) = KD^{-\alpha} + L_{\infty}$

Require: grid points $(D_i, \bar{L}_i, \hat{\sigma}_i)$ for $i = 1, \ldots, n$; a grid Λ of 400 candidate values on $[-0.5, 1.0]$

1: **for** each $\ell \in \Lambda$ with $\ell < \min_i \bar{L}_i$ **do**
2: $\quad x_i \leftarrow \log D_i, \quad y_i \leftarrow \log(\bar{L}_i - \ell), \quad w_i \leftarrow 1/\hat{\sigma}_i^2$
3: $\quad (a_\ell, \alpha_\ell) \leftarrow \arg\min_{(a,\alpha)} \sum_i w_i \left(y_i - a + \alpha x_i\right)^2$ ▷ closed form
4: $\quad \mathrm{RSS}(\ell) \leftarrow \sum_i w_i \left(y_i - a_\ell + \alpha_\ell x_i\right)^2$
5: **end for**
6: $\hat{\ell} \leftarrow \arg\min_\ell \mathrm{RSS}(\ell)$
7: **return** $\hat{K} = \exp(a_{\hat{\ell}}),\ \hat{\alpha} = \alpha_{\hat{\ell}},\ \hat{L}_\infty = \hat{\ell}$

D.2 Fitting methodology

Functional form and estimator. We fit

$$L(D) \ = \ K\, D^{-\alpha} + L_{\infty}, \tag{9}$$

with parameters (K, α, L_∞), by nonlinear weighted least squares. Because L_∞ enters nonlinearly, we solve the problem by profiling L_∞ out (Algorithm 1): for each candidate L_∞ on a 400-point grid over $[-0.5, 1.0]$, the inner problem $\log(L - L_\infty) = \log K - \alpha \log D$ is linear in $(\log K, \alpha)$ and is solved in closed form by weighted least squares on log-loss residuals, with weights $w_i = 1/\hat{\sigma}_i^2$, where $\hat{\sigma}_i$ is the sample standard error at grid point i over 3 seeds; we keep the candidate with the smallest weighted residual. Section 4.1 and this paragraph describe the same, single estimator. As an independent check, a Levenberg-Marquardt fit [21, 32] of Eq. 9 in raw space gives $(\alpha_{\mathrm{E}}, \alpha_{\mathrm{T}}, \alpha_\tau) = (0.415, 0.264, 0.107)$, within 0.006 of the reported values. On synthetic data with known ground truth, the estimator recovers α to within ± 0.015 across the relevant range, and on synthetic data generated with our exact design it recovers the correct ordering in 99.8% of replicas.

Confidence intervals. 95% CIs are computed by paired bootstrap: for each of 1000 resamples we sample 6 grid points with replacement and 3 seeds with replacement, refit, and record (K, α, L_∞). The CI for α is the range from the 2.5th to the 97.5th percentile of the bootstrap distribution. We chose the paired bootstrap over the delta method because the latter underestimates uncertainty in small-sample regimes ($n = 18$ per fit).

Goodness-of-fit diagnostics. For all per-axis fits reported in Table 2, the weighted R^2 exceeds 0.97. Residuals show no systematic pattern with D (Durbin-Watson statistic in $[1.7, 2.3]$ across all 18 fits, consistent with no autocorrelation). We also tested an alternative two-term form $L = K_1 D^{-\alpha_1} + K_2 D^{-\alpha_2} + L_\infty$; the BIC favours the single-term form (Eq. 9) at every scale we measured, indicating the data does not support a two-regime fit.

D.3 Stability of the ordering

Eight-point grids. Table 11 compares the 6-point and 8-point fits for Qwen3-8B on math+code. Adding two points per axis moves each exponent by at most 0.01, and dropping any single grid point moves it by at most 0.015.

Fit-free local slopes. A high R^2 obtained with few points and three parameters is weak evidence on its own, so we also report the ordering without any fit. Table 12 lists the local log-log slopes between adjacent points of the 8-point grids, $s_j = -[\log L(D_{j+1}) - \log L(D_j)]/[\log D_{j+1} - \log D_j]$, which involve no functional form, no L_∞, and no estimator. The three ranges do not overlap (0.36 to 0.44, 0.22 to 0.29, and 0.08 to 0.14), so every environment segment is steeper than every task segment, which is in turn steeper than every trajectory segment: 49 of 49 pairwise comparisons agree for each adjacent pair of axes (one-sided Mann-Whitney test [31], $p = 2.9 \times 10^{-4}$).

Table 11: Six-point versus eight-point fits (Qwen3-8B, math+code). The last column is the largest change in α when any single grid point is left out.

Axis	Range	α (6 points)	α (8 points)	R^2	Max. leave-one-out shift
D_{E}	4 to 384 (96×)	0.41 ± 0.03	0.40 ± 0.02	0.981	0.015
D_{T}	160 to 20,480 (128×)	0.26 ± 0.02	0.26 ± 0.02	0.984	0.011
D_τ	1 to 48 (48×)	0.11 ± 0.02	0.12 ± 0.01	0.986	0.009

Table 12: Fit-free local log-log slopes s_j between adjacent grid points of the 8-point grids (Qwen3-8B, math+code).

Segment	1	2	3	4	5	6	7
D_{E}	0.44	0.42	0.41	0.40	0.38	0.37	0.36
D_{T}	0.29	0.28	0.26	0.25	0.24	0.23	0.22
D_τ	0.14	0.13	0.12	0.11	0.10	0.09	0.08

Fewer free parameters. A joint fit with a single L_∞ shared across the three sweeps uses 7 parameters against 54 observations (47 residual degrees of freedom) and returns $(\alpha_{\mathrm{E}}, \alpha_{\mathrm{T}}, \alpha_\tau) = (0.40, 0.26, 0.11)$. Fixing L_∞ at 0, 0.10, and 0.15 instead gives 2-parameter fits of $(0.36, 0.23, 0.10)$, $(0.41, 0.26, 0.11)$, and $(0.44, 0.28, 0.12)$. The ordering holds throughout.

Differences rather than CI overlap. Non-overlapping CIs are a conservative but indirect criterion; the appropriate test is on the differences. A paired bootstrap over seeds and grid points gives $\alpha_{\mathrm{E}} - \alpha_{\mathrm{T}} = 0.148$ with 95% interval $[0.109, 0.187]$ and $\alpha_{\mathrm{T}} - \alpha_\tau = 0.150$ with 95% interval $[0.116, 0.184]$, both with $p < 0.001$.

Other model families and a search domain. Table 13 reports reduced 4-point sweeps on two further model families and on the search-agent domain, using the hyperparameters of Appendix A without retuning. The ordering holds in all three; absolute exponents on Llama-3.1-8B sit 0.02 to 0.05 below Qwen3, which is why we claim the ordering rather than the constants.

E Model-Size Exponent Fit

The main paper's law (Eq. 1) is fitted with $(D_{\mathrm{E}}, D_{\mathrm{T}}, D_\tau)$ at fixed $N = 8\mathrm{B}$. To incorporate model size into Section 6's allocation problem, we additionally fit the marginal N-axis exponent.

Protocol. Holding $(D_{\mathrm{E}}, D_{\mathrm{T}}, D_\tau)$ at saturation values for each model size, we sweep $N \in \{0.6, 1.7, 4, 8, 14, 32\}$ B (using Qwen3-0.6B/1.7B/4B/8B/14B and Qwen3-32B [53] as the largest point) and fit $L = K_N\, N^{-\alpha_N} + L_\infty$ following the same protocol as Appendix D.

Result. The fitted exponent is $\alpha_N = 0.30 \pm 0.04$ (95% CI), with $K_N = 0.85$ and L_∞ jointly fitted to the same value as the data-axis fits ($L_\infty = 0.10 \pm 0.03$). This α_N is materially smaller than Chinchilla's data-axis exponent (~ 0.5), consistent with the observation in Section 6.2 that agentic RL benefits more from data variety than from raw model scale in the regime we measured.

Range of validity. We caution that the N-sweep covers only 0.6B to 32B; extrapolation to frontier scales ($\geq$ 70B) is unverified and is listed as a limitation in Section 7 and Appendix R.

F Loss-Equivalent Metric

Definition. Standard pretraining scaling laws fit cross-entropy loss against data and parameters. RL with verifiable rewards has no smooth loss; we use

$$L \;\equiv\; -\log\big(\max(p_{\mathrm{success}}, p_0)\big), \quad p_0 = 10^{-2}. \tag{10}$$

The floor p_0 prevents divergence at zero success and corresponds to the empirical lower bound observed in our smallest-scale runs.

Why this works. The success rate p_{success} saturates at 1, producing a non-power-law plateau in raw success-rate plots. The transformation to $L = -\log p$ unfolds this plateau into a power-law decay

Table 13: Fitted exponents on further model families and a new domain (reduced 4-point sweeps).

Setting	α_{E}	α_{T}	α_{τ}
Qwen3-8B (reference, Table 2)	0.41 ± 0.03	0.26 ± 0.02	0.11 ± 0.02
Llama-3.1-8B (new family)	0.36 ± 0.05	0.23 ± 0.04	0.09 ± 0.03
Qwen2.5-7B (earlier generation)	0.38 ± 0.05	0.24 ± 0.04	0.10 ± 0.03
Search agent (new domain, Qwen3-8B)	0.42 ± 0.05	0.25 ± 0.04	0.12 ± 0.03

Table 14: Ablation of the task-deduplication threshold θ. Effective D_{T} is the count of unique tasks remaining after deduplication at each threshold; fitted α_{T} is robust across the range.

θ	Effective D_{T} at $D_{raw} = 10{,}240$	α_{T}	R^2
0.05	9,820	0.245 ± 0.02	0.974
0.10	9,310	0.254 ± 0.02	0.978
0.15 (default)	**8,460**	$\mathbf{0.261 \pm 0.02}$	**0.978**
0.20	7,210	0.268 ± 0.02	0.972
0.30	5,380	0.272 ± 0.03	0.968

matching pretraining-style scaling, allowing clean log-log fits. This is equivalent to the "effective error" transformation used in [20, 47].

Comparison to alternatives. We tested three alternative metrics on the per-axis fits: (i) raw p_{success}, (ii) logit $\mathrm{logit}(p_{\mathrm{success}}) = \log \frac{p}{1-p}$, and (iii) Brier score on per-task success. For (i), naive power-law fits saturate poorly (residuals $> 5\%$ even at $R^2 = 0.85$). For (ii), the fit is comparable in goodness ($R^2 = 0.96$ vs. our 0.97+) but the resulting exponents differ from pretraining-style α by a constant factor, complicating cross-paper comparison. For (iii), the metric is too noisy for a 3-seed regime. We thus standardize on $-\log p_{\mathrm{success}}$ throughout.

G Task Individuation: Threshold, Definition, and Human Validation

Threshold. Task counts D_{T} are deduplicated by sentence-embedding cosine distance $> \theta$ (Section 3.1); we ablate $\theta \in \{0.05, 0.10, 0.15, 0.20, 0.30\}$. Embeddings are computed with the gte-Qwen2-1.5B encoder [24]. Across this range, the fitted α_{T} varies between 0.245 and 0.272 on the math+code testbed at 8B (Table 14), with the chosen default ($\theta = 0.15$) at 0.261 ± 0.02. The qualitative ordering $\alpha_{\mathrm{E}} > \alpha_{\mathrm{T}} > \alpha_{\tau}$ is preserved across all five thresholds. At low θ (e.g., 0.05), near-duplicates inflate D_{T} but contribute little gradient information, slightly biasing the fitted α downward. At high θ (e.g., 0.30), the deduplication is so aggressive that we remove genuinely distinct tasks, slightly biasing it upward. The plateau at $\theta \in [0.10, 0.20]$ is robust, justifying our default choice.

Definition. The threshold ablation tests numerical sensitivity but not the definition itself, which is the sharper question: semantically different instructions can require nearly identical skills, while superficially similar ones can require different reasoning or tool-use patterns. We therefore refit α_{T} under five alternative individuation rules, two of them embedding-free (Table 15): the raw count without deduplication; the default rule with a different encoder (e5-mistral-7b [49]); a skill signature that deduplicates by the tool-call skeleton (tool types plus argument types) of a reference solution; a verifier signature that deduplicates by the structure of the verification program; and a human-validated pool. All six values lie in $[0.238, 0.261]$, strictly between α_{τ} and α_{E} under every rule.

Human validation. We sampled 500 task pairs stratified by embedding distance, and three annotators, volunteers from MBZUAI who participated without compensation, labeled each pair for whether it requires a materially different skill or tool-use pattern (Fleiss' kappa 0.68 [8]). Agreement with the majority label is 87.2% for the skill signature (Cohen's kappa 0.74 [5]), 84.6% for the verifier signature (kappa 0.69), 80.4% for the embedding rule (kappa 0.61, with 11.2% false merges), and 68.0% for the raw count (kappa 0.36). The embedding rule is substantial but imperfect and the skill signature is better, so we use the skill-signature refit as the primary robustness check and keep the embedding rule as the cheap operational default. The ordering does not depend on which rule is used.

Table 15: Fitted task exponent under alternative definitions of a distinct task (Qwen3-8B, math+code). Effective D_{T} is the number of distinct tasks at $D_{raw} = 10{,}240$.

Rule for a distinct task	Effective D_{T}	α_{T}
Raw count, no deduplication	10,240	0.238 ± 0.02
Embedding distance > 0.15, gte-Qwen2-1.5B (default)	8,460	0.261 ± 0.02
Embedding distance > 0.15, e5-mistral-7b	8,310	0.257 ± 0.02
Skill signature (reference tool-call skeleton)	6,120	0.243 ± 0.03
Verifier signature (verification-program structure)	6,980	0.251 ± 0.02
Human-validated pool	6,540	0.248 ± 0.03

H Reward Density as a Control Variable

The main paper treats D_{R} as a control variable with two categories (sparse outcome reward and dense process reward) and does not claim a reward scaling law comparable to those of the three cardinality axes. Here we report its marginal sensitivity under a continuous treatment via the number of process-reward signals per trajectory.

Protocol. We instantiate process rewards using a Qwen3-8B-based PRM trained following [25] on a held-out math+code corpus. For each trajectory, we extract $k \in \{0, 1, 3, 8\}$ process-reward signals (placed at uniformly spaced positions across the trajectory length) and append them as additional return-to-go terms. $k = 0$ recovers sparse outcome reward; $k = 8$ provides dense step-level supervision.

Marginal sensitivity to reward density. Fitting $L = K_R(k+1)^{-\alpha_R} + L_\infty$ on the math+code testbed at 8B yields

$$\alpha_R = 0.18 \pm 0.03, \quad K_R = 0.41, \quad L_\infty = 0.11. \tag{11}$$

We report α_R as the marginal sensitivity of the loss to reward density on this testbed, not as a scaling exponent on par with those of the cardinality axes; numerically, it falls between the trajectory ($\alpha_\tau \approx 0.11$) and task ($\alpha_{\mathrm{T}} \approx 0.26$) exponents.

Implications. Reward density and trajectory count interact. Holding the total reward-signal budget $k \cdot D_\tau$ fixed, we observe approximate substitutability: doubling k at half D_τ produces equivalent loss within ± 0.5 pp on the same compute. This justifies treating D_{R} as an effective multiplier on D_τ in Section 6.2's allocation, rather than as an independent axis. We leave a fully orthogonal treatment of reward density to future work.

I Synthesis Discount Estimation and Common Harness

Estimator. For each (synthesis method, axis) pair we hold total $D = D_r + D_s$ constant and sweep the ratio $r = D_s/D_r \in \{0, 0.25, 1, 4, 16\}$, fitting $\kappa(r = 1)$ from

$$L(D_r, D_s) \;=\; f\big(D_r + \kappa\, D_s\big), \tag{12}$$

where f is the fitted single-axis law of Section 4. Since f is monotone, this gives a closed-form estimator for κ at each r:

$$\hat{\kappa}(r) \;=\; \frac{f^{-1}\big(\hat{L}(D_r, D_s)\big) - D_r}{D_s}. \tag{13}$$

We report $\hat{\kappa}(r = 1)$ in Table 3 as the headline number. The collapse curves of Figure 4 use a denser ratio grid (Appendix J).

Errors. Bootstrap CIs for κ propagate uncertainty in both $\hat{L}$ and the parameters of f (Appendix D). Reported half-widths are 95% bootstrap CIs over 1000 paired resamples.

Common harness. Table 3 uses one common harness rather than numbers reported in the original papers. All nine methods use math+code with Qwen3-8B and the same 3 seeds; the same real pool of 10,240 tasks and 256 environments; the same generator (Qwen3-32B [53], replacing each method's original generator), with 8 generation calls per unit and at most 4k output tokens; identical executability and verifier filtering and deduplication thresholds; the same outcome verifier; identical

Table 16: Discount and collapse ratio of each single-axis method as reported in Table 3 and after reduction to a common generate, filter, and train core.

Method	Axis	κ (Table 3)	κ (common core)	$r^\star$ (Table 3)	$r^\star$ (common core)
AWM	D_{E}	0.71	0.68 ± 0.05	9.4	8.7 ± 1.0
GenEnv	D_{E}	0.66	0.65 ± 0.06	8.1	7.9 ± 1.1
AutoPlay	D_{T}	0.54	0.55 ± 0.04	3.1	3.2 ± 0.5
SCA	D_{T}	0.51	0.52 ± 0.05	2.8	2.9 ± 0.5
RLVE	D_{T}	0.57	0.56 ± 0.04	3.4	3.3 ± 0.5
SWiRL	D_τ	0.34	0.36 ± 0.06	1.4	1.5 ± 0.3
TDScaling	D_τ	0.42	0.40 ± 0.05	1.7	1.6 ± 0.3
Self-Play	D_τ	0.31	0.33 ± 0.06	1.2	1.3 ± 0.3

Table 17: Collapse ratio of the three representative synthesis methods (Qwen3-8B, math+code) under alternative definitions of collapse.

Criterion	$r^\star_{\mathrm{E}}$ (AWM)	$r^\star_{\mathrm{T}}$ (AutoPlay)	$r^\star_\tau$ (SWiRL)
Crossing of 0.95 by the fitted logistic (default)	9.4	3.4	1.4
Crossing of the real-only baseline (1.00)	8.5	4.0	1.1
Peak of the curve	2.4	1.1	0.4

downstream training (1,000 steps, batch 256, $G = 8$, and the same held-out evaluation); and the same ratio grid and κ estimator.

Common-core ablation. Because the selected methods also differ in maturity and implementation quality, we strip method-specific components (e.g., AWM's repair loop, AutoPlay's exploration bonus, and DreamGym's replay buffer), leaving a common generate, filter, and train core (Table 16). Across the eight single-axis methods, axis membership explains 97% of the variance of common-core κ (one-way ANOVA, $F(2, 5) = 76.6$, $p < 0.001$), with within-axis spread at most 0.07 against 0.30 between axes, so the axis effects are unlikely to be method artifacts. The exponents of Section 4 are measured independently on axis-controlled pools; Table 3 only supplies κ_i to Section 6. Because κ_i enters the allocation problem only through the ratio c_i/κ_i, the cost perturbations of Appendix N also cover the corresponding misestimates of κ_i.

J Collapse-Ratio Estimation

Definition and grids. We define $r^\star$ as the ratio at which normalized held-out success, past its peak, falls 5% below the real-only value at $r = 0$. Two ratio grids are used and are kept separate: κ is estimated on $r \in \{0, 0.25, 1, 4, 16\}$ (Appendix I), while the collapse curves of Figure 4 use $r \in \{0, 0.1, 0.25, 0.5, 1, 2, 3, 4, 6, 8, 10, 16, 25\}$, and every curve is normalized to the real-only value of the same seed group.

Estimator. We fit a monotone four-parameter logistic in $\log r$ to the post-peak branch of each curve, weighted by $1/\hat{\sigma}^2$, and take $r^\star$ as its analytic crossing of 0.95; confidence intervals use the paired bootstrap of Appendix D. This gives $r^\star_{\mathrm{E}} = 9.4$, $r^\star_{\mathrm{T}} = 3.4$, and $r^\star_\tau = 1.4$ for AWM, AutoPlay, and SWiRL. Relative to Table 3, only AutoPlay moves (3.4 against 3.1, within its CI). Table 17 shows that the ordering does not depend on the criterion used to define collapse.

Mechanism: groundedness of synthesis. Section 5.2 attributes the ordering $r^\star_E \gg r^\star_T \gg r^\star_\tau$ to how strongly each axis is grounded by external truth. We additionally quantify this: across the three synthesis methods we measured the distributional drift between synthetic and real samples by the maximum mean discrepancy (MMD) [12] of state-action embeddings against the reference real-data distribution. MMD scales with r as $\mathrm{MMD}(r) \propto r^{0.45}$ for environment synthesis, $r^{0.62}$ for task synthesis, and $r^{0.91}$ for trajectory synthesis on Qwen3-8B. The exponents inversely track $r^\star$: faster drift implies earlier collapse, a quantitative confirmation of the groundedness story. Replicating the measurement on Llama-3.1-8B gives drift exponents of 0.47, 0.64, and 0.88, so the mechanism that predicts the ordering transfers across model families.

K Lagrangian Solution to The Allocation Problem

Setup. The constrained optimization (Eq. 6) is

$$\min_{N,D_{\mathrm{E}},D_{\mathrm{T}},D_\tau} \quad \sum_{i\in\mathcal{A}} K_i\, D_i^{-\alpha_i} + K_N\, N^{-\alpha_N} + L_\infty \quad \text{s.t.} \quad \sum_{i\in\mathcal{A}} \frac{c_i}{\kappa_i} D_i + c_N N = C, \tag{14}$$

where $\mathcal{A} = \{\mathrm{E}, \mathrm{T}, \tau\}$ and the constraint is taken with equality at the optimum (since the objective is strictly decreasing in each D_i).

KKT conditions. Form the Lagrangian $\mathcal{L} = \text{(Eq. 14)} + \lambda \cdot \big[\sum_i \frac{c_i}{\kappa_i} D_i + c_N N - C\big]$. Setting $\partial\mathcal{L}/\partial D_i = 0$ for each axis:

$$-\alpha_i K_i\, D_i^{-\alpha_i - 1} + \lambda \frac{c_i}{\kappa_i} = 0 \quad \Longrightarrow \quad D_i^\star \;=\; \left(\frac{\alpha_i K_i\, \kappa_i}{\lambda\, c_i}\right)^{\frac{1}{\alpha_i+1}}. \tag{15}$$

Analogously for N:

$$N^\star \;=\; \left(\frac{\alpha_N K_N}{\lambda\, c_N}\right)^{\frac{1}{\alpha_N+1}}. \tag{16}$$

Closed-form scaling in C. Substituting Eqs. 15 and 16 into the budget equation gives, after collecting powers of λ:

$$\sum_{i\in\mathcal{A}} \frac{c_i}{\kappa_i} \left(\frac{\alpha_i K_i \kappa_i}{\lambda c_i}\right)^{\frac{1}{\alpha_i+1}} + c_N \left(\frac{\alpha_N K_N}{\lambda c_N}\right)^{\frac{1}{\alpha_N+1}} \;=\; C. \tag{17}$$

For each axis the exponent of λ is $-1/(\alpha_i+1)$, so to leading order $\lambda \propto C^{-\xi}$ for some effective exponent ξ determined by the dominant axis. Substituting back gives

$$D_i^\star \;\propto\; C^{\,\xi/(\alpha_i+1)}, \qquad N^\star \;\propto\; C^{\,\xi/(\alpha_N+1)}. \tag{18}$$

Numerical solution. With our fitted values ($\alpha_{\mathrm{E}} = 0.41$, $\alpha_{\mathrm{T}} = 0.26$, $\alpha_\tau = 0.11$, $\alpha_N = 0.30$; $K_E = 3.39$, $K_T = 0.53$, $K_\tau = 0.38$, $K_N = 0.85$; $\kappa_E = 0.71$, $\kappa_T = 0.54$, $\kappa_\tau = 0.34$; c_E, c_T, c_τ, c_N from Appendix L), we solve the system numerically by binary search on λ to satisfy the budget constraint. Repeating across $C \in \{1, 2, 4, 8, 16, 32\}\, C_0$ and fitting power-laws to the resulting $D_i^\star(C)$ yields the recipe in Eq. 8: $D_{\mathrm{E}}^\star \propto C^{0.34}$, $D_{\mathrm{T}}^\star \propto C^{0.27}$, $D_\tau^\star \propto C^{0.13}$, $N^\star \propto C^{0.26}$. The four exponents sum to approximately one ($0.34 + 0.27 + 0.13 + 0.26 = 1.00$).

L Cost-Coefficient Calibration and Refined Cost Models

Decomposition. The cost coefficients (c_E, c_T, c_τ, c_N) in Eq. 5 aggregate two physically distinct cost sources: **(i) synthesis cost** (one-time generation of environments, tasks, or trajectories), measured in API/compute USD per unit; and **(ii) training cost** (rollout + gradient FLOPs), measured in H100-GPU-hours per unit. We normalize both into a common base unit C_0.

Calibration to C_0. We define $C_0 = 2.4 \times 10^{18}$ FLOPs as the cost of one Qwen3-4B GRPO training run on the math+code testbed for 1000 gradient steps, which corresponds approximately to 5 H100-GPU-hours at the throughput we measured on our cluster (vLLM 0.6.3 + FSDP). Synthesis costs are converted via 1 H100-hour ≈ \$2.50 (a representative cloud rate at the time of writing); the resulting normalized coefficients are:

$$c_E = 3.0\times10^{-2}, \quad c_T = 1.0\times10^{-3}, \quad c_\tau = 3.0\times10^{-3}, \quad c_N = 1.5\times10^{-1}. \tag{19}$$

The relative magnitudes carry the meaning, not the absolute USD values: $c_E/c_T \approx 30$ reflects that environment synthesis (4.10 USD/env) is much costlier per unit than task synthesis (0.18 USD/task), and $c_N/c_\tau \approx 50$ reflects that doubling model size is much more expensive than doubling trajectory count when the rest is held fixed.

Multiplicative compute term. A more accurate cost model accounts for the fact that rollout and gradient FLOPs scale jointly with D_τ and N:

$$C_{\mathrm{train}}(D_\tau, N) \;=\; \big(c_{\mathrm{rollout}} \cdot H + c_{\mathrm{update}}\big) \cdot N \cdot D_\tau, \tag{20}$$

Table 18: Measured cost drivers per domain: average horizon H (steps), rollout tokens per trajectory, environment latency per call, verifier time per trajectory, failed-rollout fraction f, and accelerator utilization η.

Domain	H	Rollout tokens/traj	Env latency/call	Verifier/traj	f	η
Math+code	3.4	2.6k	0.9 s	0.4 s	0.07	0.61
SWE	18.2	11.4k	6.3 s	41 s	0.19	0.38
Web	12.7	7.9k	2.4 s	1.8 s	0.14	0.44
UI/tool	9.6	5.2k	1.1 s	0.9 s	0.11	0.52

Table 19: Recipe exponents under progressively richer cost models (math+code coefficients unless stated otherwise).

Cost model	D_{E} exp.	D_{T} exp.	D_τ exp.	N exp.
Linear-additive (Eq. 5)	0.34	0.27	0.13	0.26
+ bilinear rollout term $c \cdot N \cdot D_\tau$	0.33	0.27	0.14	0.26
+ tool and environment latency, verifier cost	0.31	0.28	0.15	0.26
+ failed-rollout overhead $1/(1-f)$	0.30	0.28	0.16	0.26
+ batching and utilization η	0.30	0.28	0.16	0.26
Full model, SWE coefficients (worst case for D_{E})	0.28	0.29	0.18	0.25
Full model, web coefficients	0.33	0.27	0.14	0.26

where H is the average rollout length. We verified that absorbing this bilinear term into an effective $c_\tau \cdot N + c_N$ (linearised at the optimum) gives KKT optima that differ from Eq. 8 by less than 0.02 in every exponent across $C \in [1, 32]\, C_0$, justifying the simpler additive form in the main text.

Measured cost drivers. Rollout cost also depends on horizon length, tool latency, environment execution, verifier cost, failed rollouts, and infrastructure utilization. We instrumented the training stack and report these drivers per domain in Table 18.

Refined cost models. Table 19 re-derives the recipe adding one factor at a time: the bilinear rollout term $c \cdot N \cdot D_\tau$ of Eq. 20, tool and environment execution latency plus verifier cost, failed-rollout overhead $1/(1-f)$, and batching and utilization η. Every refinement moves budget from environments toward tasks and trajectories, but the largest shift in any exponent is 0.06, which changes predicted held-out success at $C = 32C_0$ by 0.7 points against the measured 8.3-point gap over the naive allocation. The qualitative conclusion is therefore not near its flipping point, although the precise exponents are stack-dependent. In the SWE worst case (18-step horizons, a 41 s verifier, and 19% failed rollouts), the environment and task exponents become statistically indistinguishable, so we do not claim that D_{E} strictly dominates everywhere.

Break-even condition. The environment axis loses its top rank only when the fully loaded cost per environment unit exceeds roughly $3.4\times$ our highest measured value (11.8 USD-equivalent per environment, on SWE) at unchanged trajectory cost. Across four domains and two clusters (H100 with vLLM and FSDP, and an A100 stack), the measured fully loaded cost ranges from 2.9 to 11.8 USD-equivalent per environment, so practitioners can check their own infrastructure against this number rather than trusting ours. Section 6.4 applies the same break-even to the acquisition of new worlds.

M Cross-Domain Robustness of The Recipe

We re-derive the recipe (Eq. 8) on each of the four domains independently to verify generality.

As Table 20 shows, the web and UI/tool domains have slightly larger D_{E} exponents and smaller D_τ exponents than math+code, consistent with these domains' richer state-space structure (more API surface to generalize across). The SWE domain is an outlier in the opposite direction: its D_{E} exponent is the smallest because the code-execution sandbox provides relatively limited environmental variation between tasks. Despite these shifts, the qualitative ordering and the approximate normalization

Table 20: Recipe exponents per domain. The qualitative ordering $D_{\mathrm{E}}^{\star} > D_{\mathrm{T}}^{\star} > D_{\tau}^{\star}$ is preserved in every domain, and absolute exponents shift by less than ± 0.04 from the math+code reference.

Domain	D_{E} exp.	D_{T} exp.	D_{τ} exp.	N exp.	$\sum$
Math+Code (default)	0.34	0.27	0.13	0.26	1.00
SWE	0.31	0.28	0.15	0.27	1.01
Web	0.36	0.27	0.12	0.25	1.00
UI/Tool	0.37	0.26	0.11	0.27	1.01
Range across domains	± 0.03	± 0.01	± 0.02	± 0.02	n/a

Table 21: Recipe-exponent sensitivity to $\pm 50\%$ perturbations of cost coefficients (math+code, 8B). All shifts are < 0.05 on every axis.

Perturbation	D_{E} exp.	D_{T} exp.	D_{τ} exp.	N exp.
Default	0.340	0.270	0.130	0.260
$c_E \times 1.5$	0.305	0.291	0.142	0.262
$c_E \times 0.5$	0.388	0.244	0.115	0.253
$c_T \times 1.5$	0.348	0.243	0.144	0.265
$c_T \times 0.5$	0.328	0.305	0.114	0.253
$c_\tau \times 1.5$	0.346	0.275	0.111	0.268
$c_\tau \times 0.5$	0.331	0.264	0.156	0.249
$c_N \times 1.5$	0.349	0.278	0.135	0.238
$c_N \times 0.5$	0.327	0.260	0.124	0.289
Max abs. shift	**0.048**	**0.035**	**0.026**	**0.029**

$\sum_i \exp_i \approx 1$ are preserved everywhere. Under the richer cost models of Appendix L, however, the SWE environment and task exponents become indistinguishable.

N Cost-Coefficient Sensitivity

To verify the recipe is not over-fit to our specific cost calibration (Appendix L), we perturb each coefficient $\pm 50\%$ and re-solve the Lagrangian.

The largest sensitivity is to c_E (the environment-synthesis cost), which makes sense intuitively: c_E sets the price of the highest-leverage axis, and small changes propagate. Even so, all perturbations stay within ± 0.05 of the default exponents, and the qualitative ordering is preserved in every row of Table 21. Because each κ_i enters the allocation problem only through c_i/κ_i, these perturbations also cover the corresponding misestimates of the synthesis discounts. We conclude the recipe is robust to reasonable mis-specification of cost coefficients; Appendix L complements these perturbations with structurally richer cost models.

O Negative Results and Failed Fits

For transparency we report alternatives we tried that did not work, in the spirit of strengthening the paper's positive claims.

Multiplicative interaction term. We tested $L = K_E D_{\mathrm{E}}^{-\alpha_{\mathrm{E}}} \cdot K_T D_{\mathrm{T}}^{-\alpha_{\mathrm{T}}} \cdot K_\tau D_\tau^{-\alpha_\tau} + L_\infty$ (fully multiplicative form). Goodness-of-fit was strictly worse than the additive form (R^2 drops from 0.978 to 0.951 on the math+code fit), and the resulting individual exponents were not robust across model sizes (± 0.08 shift between 4B and 14B). We retain the additive form.

Unified data axis. A reasonable null hypothesis is that the three cardinality axes can be collapsed into a single "effective data" axis $D_{\text{eff}} = w_E D_{\mathrm{E}} + w_T D_{\mathrm{T}} + w_\tau D_\tau$ with weights w_i fitted jointly. Best-fit weights are $(w_E, w_T, w_\tau) = (4.2, 1.0, 0.04)$, but the resulting one-axis fit has $R^2 = 0.83$ vs. 0.978 for the three-axis decomposition, and the weights themselves are unstable across domains. We conclude the axes are genuinely independent and not a relabeling of one underlying quantity.

Direct success-rate fit (no log transform). We tried fitting power laws directly in p_{success} space rather than the loss-equivalent L (Appendix F). Fits saturate poorly near the success-rate ceiling, with residuals exceeding 5% at the high-success grid points and driving R^2 below 0.85. The log transform is necessary.

Curriculum at fixed allocation. We tested whether scheduling difficulty (curriculum) within a fixed allocation could substitute for axis decomposition. A simple difficulty-easy-to-hard curriculum [3] on the naive allocation gives a 1.2 pp improvement at $C = 32$ but does not approach our recipe's 8.3 pp gain over naive. Curriculum is a useful but orthogonal improvement.

P Reproducibility

Code and data release. We release, at `https://github.com/BokwaiHo/AgentRL_Scaling`: (i) the verl-compatible training scripts for all per-axis sweeps; (ii) the calibrated $(K_i, \alpha_i, \kappa_i, c_i)$ fitting code, including the profiled estimator of Algorithm 1; (iii) the synthesised env/task corpora used (subject to licensing of upstream data); (iv) plotting code reproducing every figure in the paper; (v) the per-grid-point environment signature manifests and the structural-factor inventory of Appendix C; and (vi) the held-axis calibration script together with its pre-specified criterion (Appendix A).

Determinism. All RL runs use deterministic torch settings (`torch.use_deterministic_algorithms(True)`) and fixed seeds. Run-to-run variance reported in tables is genuinely the seed effect, not stochastic data ordering.

Hyperparameter sweep budget. We did not tune RL hyperparameters per scaling-law fit. The hyperparameters (Appendix A) were fixed once based on a held-out 8B/math+code calibration and used unchanged across all cells of Table 2, all 4 cross-domain re-fits, the additional model families and search domain, and all sensitivity analyses. This is critical for the validity of the scaling-law fits: had we re-tuned hyperparameters per cell, the fitted α_i would conflate data-axis effects with hyperparameter effects.

Q Compute Disclosure

The total experimental compute for this paper is approximately $61{,}000$ H100-GPU-hours: approximately $48{,}000$ for the experiments of the original study and approximately $13{,}000$ for the additional experiments added in this version. The breakdown of the former is:

- Section 4 per-axis sweeps and robustness (Table 2): $\sim$32,000 GPU-hours (18 cells $\times$ 6 grid points $\times$ 3 seeds, with median per-run cost varying by domain).
- Section 5 synthesis-discount and collapse experiments (Fig. 4, Table 3): $\sim$10,000 GPU-hours.
- Section 6 IsoFLOP sweeps and held-out validation: $\sim$5,000 GPU-hours, dominated by the single Qwen3-14B held-out run at $C = 32C_0$.
- Failed runs, hyperparameter calibration, and pilots: $\sim$1,000 GPU-hours.

The synthesis-side compute (env/task generation via AWM, AutoPlay, etc.) used an additional $\sim$3,500 H100-GPU-hours plus approximately $2,800 in API costs for the closed-source generators we used as baselines. The additional experiments, reported in Appendices B to D, G, I, J, and L (the eight-point grid extension, the training-signal conventions, the signature-restricted, placebo, and world-structure sweeps, the coverage pools, the further model families and search domain, the common-core synthesis ablation, and the refined cost models), ran on the same infrastructure and used approximately $13{,}000$ H100-GPU-hours in total.

R Limitation Analysis

This work has several limitations. First, although we evaluate the proposed axis-decomposed scaling framework across multiple model sizes, algorithms, domains, and model families, the experiments remain limited to the specific model families, benchmark environments, and compute scales studied in this paper. In particular, the model-size sweep covers open-weight models in the sub-frontier regime, and the held-out validation is performed at a single larger scale. Llama-3.1-8B and Qwen2.5-7B

preserve the ordering of the exponents while shifting their absolute values by up to 0.05, and the fitted exponents and compute-optimal allocation recipe may change at frontier scale, under different pretraining distributions, or for agents with substantially different tool-use and reasoning priors. Therefore, the ordering is our claim, and the numerical exponents reported in this paper should be viewed as empirical estimates for the measured regime rather than universal constants.

Second, our decomposition of agentic data into task, environment, and trajectory axes plus a reward-density control variable is a useful abstraction, but it is not perfectly orthogonal. In practice, tasks and environments often co-vary: a new environment may induce new task types, and a new task distribution may require different environment structure; our sweeps re-bind a fixed task pool to compatible environments (Appendix B), but only part of the task $\times$ environment grid is realizable. Similarly, reward density can interact with trajectory diversity because denser rewards may change which rollouts are informative. We model some cross-axis interactions, especially between task and environment diversity, but richer higher-order interactions may exist. Future work should study more expressive interaction terms and determine when the additive power-law approximation breaks down.

Third, the resolution of the environment axis into world and structure diversity rests on limited world counts. World diversity is swept over only 1 to 8 real worlds on SWE and UI/tool and 1 to 4 on web (1 to 11 repositories in the SWE-only sweep), so α_{W} spans a single order of magnitude. The structure axis has a finite realizable ceiling under our factor inventory. The structural exponent α_{V} varies across domains from 0.28 to 0.41, and we do not offer a single value. We do not claim that structure and world variation are interchangeable, and we do not claim that any of the fitted relations continue past the measured ranges.

Fourth, the loss-equivalent metric used in this paper is based on held-out task success. This makes scaling-law fitting practical for verifiable-reward RL, but it does not capture every property relevant to agent quality. For example, two agents with similar success rates may differ in robustness, calibration, latency, safety, interpretability, or the severity of their failures. The metric also depends on the coverage and difficulty distribution of the held-out evaluation set. As a result, improvements predicted by the fitted law should not be interpreted as comprehensive improvements in all downstream deployment settings.

Fifth, our treatment of synthetic data quality is necessarily approximate. The synthesis discount factor κ and collapse ratio $r^\star$ provide a common way to compare methods, but they compress many dimensions of quality into a small number of scalar quantities. Synthetic environments, tasks, and trajectories may differ in realism, diversity, difficulty, distributional coverage, and susceptibility to model-generated artifacts. These factors may vary across domains and over training time. In addition, the measured discount for a synthesis method depends on the base model, generator, verifier, and filtering pipeline used in the experiment; we fix these through a common harness (Appendix I), but different synthesis pipelines may yield different effective discounts even when targeting the same axis.

Sixth, the compute-allocation recipe relies on a simplified cost model. We assume an approximately linear, axis-additive cost structure with per-axis cost coefficients and synthesis discounts. Real training systems may have non-linear bottlenecks due to rollout latency, tool execution, environment simulation, distributed training overhead, memory constraints, data storage, or human review costs. Adding the main measured drivers changes recipe exponents by at most 0.06 but makes the environment and task exponents indistinguishable under SWE coefficients (Appendix L). The recipe is therefore best understood as a principled first-order allocation rule rather than an exact prescription for every infrastructure setting. Practitioners should recalibrate the cost coefficients and fitted exponents under their own hardware, software stack, and data-generation pipeline before using the recipe operationally.

Seventh, our experiments focus on benchmark-style domains with controlled evaluation and relatively clear reward signals. Many real-world agentic tasks involve ambiguous objectives, partial observability, long-horizon dependencies, dynamic user preferences, and non-stationary environments. In such settings, defining task success and reward density is more difficult, and synthetic data may introduce artifacts that are harder to detect. Extending the framework to open-ended real-world applications will require stronger evaluation protocols, safety checks, and methods for measuring distribution shift.

Finally, this work does not provide a complete safety or deployment framework for agentic systems. The proposed scaling laws can make agentic RL training more efficient, but they do not guarantee that the resulting agents are safe, fair, private, secure, or aligned with user intent. These concerns require separate evaluations and safeguards. Future work should combine axis-level data scaling with robustness testing, adversarial evaluation, privacy analysis, fairness auditing, and deployment-time monitoring.

S Ethics Statement

This work is an empirical study of data scaling and synthesis allocation in agentic reinforcement learning. The experiments are conducted on benchmark-style math, code, web, UI/tool, and search environments, using scripted training pipelines and verifiable or process-based reward signals. We do not intentionally collect or use private personal data, and the synthetic tasks, environments, and trajectories studied in this paper are intended for research evaluation rather than for deployment in real user-facing decision systems. To reduce privacy risk, experimental logs should be stored securely, inspected for unintended sensitive content where applicable, and released only after appropriate filtering or anonymization.

The work involves synthetic data generation along multiple axes, including tasks, environments, and trajectories. Synthetic data can amplify biases, encode unrealistic assumptions, or produce artifacts that affect downstream model behavior. Our measurements focus on aggregate scaling behavior and task success, and they should not be interpreted as evidence that a trained agent is safe, fair, or reliable in deployment. Any use of the proposed allocation recipe in real applications should include additional evaluations for robustness, bias, privacy leakage, security, and subgroup-level performance.

Because agentic systems can interact with tools, websites, code execution environments, or simulated applications, there are potential dual-use risks. More efficient agentic RL training could improve beneficial automation, but it could also lower the cost of training agents for harmful activities such as phishing, spam, vulnerability discovery, unauthorized data collection, or manipulation of online services. Our experiments should be run in sandboxed environments with controlled tool access, and released artifacts should avoid enabling direct misuse. We encourage practitioners to pair these techniques with access controls, abuse monitoring, rate limits, permission boundaries, and adversarial safety evaluations.

We also consider fairness and labor-related issues. Apart from a small annotation of task-pair similarity by three volunteer annotators from MBZUAI who participated without compensation (Appendix G), which labels properties of synthetic tasks rather than of the annotators themselves, the paper does not rely on new human-subject experiments or sensitive demographic annotations. If future extensions use human annotators, user interaction traces, or real-world application data, they should follow informed-consent procedures, provide appropriate compensation, minimize exposure to harmful content, and comply with applicable institutional and legal requirements. Similarly, if agentic systems trained using these methods are deployed in socially consequential domains, humans should retain meaningful oversight and affected users should have mechanisms for appeal or correction.

Overall, this paper provides a framework for measuring and allocating agentic RL data, not a guarantee of responsible deployment. The ethical use of this work depends on the context in which the resulting agents are trained and applied. We recommend that downstream users treat the proposed scaling laws and synthesis strategies as one component of a broader responsible-development process that includes privacy protection, security review, fairness auditing, misuse prevention, and transparent reporting of data sources and evaluation limitations.

T Broader Impacts

This work studies how data scales in agentic reinforcement learning and proposes a framework for allocating synthesis budget across tasks, environments, trajectories, and reward density. The potential positive societal impact is that more efficient data allocation can reduce the compute and financial cost of training capable agentic systems. By identifying which forms of data provide the highest marginal return, the proposed framework may help researchers and practitioners train useful agents with fewer resources, improving accessibility for academic labs and smaller organizations. More efficient agentic RL could also accelerate progress on beneficial applications such as software

engineering assistance, scientific problem solving, education, accessibility tools, and automation of repetitive digital workflows. In addition, the explicit decomposition of agentic data may improve transparency: rather than treating training data as an undifferentiated token budget, our framework encourages practitioners to document what kinds of tasks, environments, trajectories, and reward signals are used.

At the same time, the same efficiency gains could enable harmful or unintended uses. More capable and cheaper-to-train agents may be misused for automated disinformation campaigns, generation of fake online profiles, scalable phishing or social engineering, spam, scraping, surveillance, or other forms of cyber abuse. Because our results suggest that environment and task synthesis can be especially high-leverage, a malicious actor could use similar methods to generate many realistic interaction settings for training agents to operate across websites, applications, or tool interfaces. The work therefore should not be interpreted as an endorsement of unrestricted deployment of agentic systems. Practical use of these techniques should be accompanied by safety evaluations, access controls, monitoring for abuse, and domain-specific restrictions where agent actions can affect real users or systems.

There are also fairness and representational concerns. Synthetic tasks and environments may encode the assumptions, biases, or omissions of the systems and datasets used to generate them. If agentic RL systems trained under such distributions are deployed in settings involving hiring, lending, education, healthcare, policing, or access to public services, they could make or support decisions that unfairly affect particular demographic or socioeconomic groups. Our scaling laws measure aggregate task success and do not by themselves guarantee equitable performance across subpopulations, languages, regions, or accessibility needs. Any downstream deployment should therefore include disaggregated evaluation, bias auditing, human oversight, and clear limits on the decisions agents are allowed to make autonomously.

Privacy is another important consideration. Agentic environments often involve realistic user data, application states, logs, tool outputs, or simulated personal information. Even when data is synthetic, it may be derived from real data sources or may resemble sensitive user records. Care is required to avoid memorization, leakage, or reconstruction of private information through generated tasks, environments, trajectories, or reward traces. Practitioners applying this framework should use data minimization, anonymization where appropriate, secure storage of trajectories and logs, and privacy reviews for any environment that models real users or organizations.

Finally, security risks arise because agentic RL systems interact with tools and external environments. Training agents to generalize across diverse environments may improve their usefulness, but it may also improve their ability to discover vulnerabilities, bypass interface constraints, or execute unintended actions. Synthetic environments should therefore be sandboxed, tool permissions should be least-privilege, and evaluations should include adversarial and misuse-oriented testing. We view our contribution as a measurement and allocation framework, not as a complete safety framework. Future work should integrate the proposed data-scaling perspective with stronger safeguards for alignment, robustness, privacy, security, and responsible deployment.

U Declaration of LLM usage

Large language models are a central object of study and an operational component of this work. Our experiments train and evaluate different-family base models under agentic reinforcement learning, using LLM policies to generate multi-step trajectories involving tool calls, environment feedback, and verifiable rewards. In addition, several data-synthesis procedures studied in the paper operate along LLM-agentic data axes: environment synthesis, task synthesis, and policy-rollout-based trajectory generation. These uses are part of the scientific methodology rather than merely writing, editing, or formatting assistance.

All reported scaling-law fits, synthesis-discount estimates, collapse-ratio measurements, allocation recipes, and held-out validations are computed from logged experimental runs and scripted analyses. LLM outputs used during synthesis or rollout are treated as experimental data and are evaluated through the verification and reward mechanisms described in the paper. The authors remain responsible for the design of the experiments, interpretation of results, and all claims made in the paper. Any use of general-purpose LLM tools for language polishing or formatting did not affect the core methodology, experimental results, or scientific conclusions.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: Please see Section 1.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: Please see Appendix R.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [Yes]

Justification: Please see Section 6 and Appendix K.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: Please see Section 3.2, Appendix A and Appendix P.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
   - If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
   - Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
   - While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
     (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
     (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
     (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
     (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: Please refer to our provided Github repository `https://github.com/BokwaiHo/AgentRL_Scaling`.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: Please see Section 3.2 and Appendix A.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [Yes]

   Justification: Please see Section 3.2, Section 4, and Section 5.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
   - The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
   - The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
   - The assumptions made should be given (e.g., Normally distributed errors).
   - It should be clear whether the error bar is the standard deviation or the standard error of the mean.

- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: Please see Appendix A and Appendix Q.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: Please see Appendix S.

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [Yes]

    Justification: Please see Appendix T.

    Guidelines:

    - The answer [N/A] means that there is no societal impact of the work performed.
    - If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
    - Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.

- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [N/A]

    Justification: This paper poses no such risks.

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
    - Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
    - We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [Yes]

    Justification: Please see Section 3 and Appendix A.

    Guidelines:

    - The answer [N/A] means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.
    - The name of the license (e.g., CC-BY 4.0) should be included for each asset.
    - For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
    - If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
    - For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.

- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [Yes]

    Justification: Please check our provided Github repository `https://github.com/BokwaiHo/AgentRL_Scaling` for code assets and corresponding document.

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

    Justification: The paper involves neither crowdsourcing nor research with human subjects. Its only human input is a small annotation of 500 synthetic task pairs by three volunteer annotators from MBZUAI, who participated without compensation and labeled properties of the tasks rather than of themselves; the labeling question and the agreement statistics are reported in Appendix G.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
    - According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: This paper does not involve crowdsourcing nor research with human subjects.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.

- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: [Yes]

    Justification: Please see Appendix U.

    Guidelines:

    - The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
    - Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.